

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Thiết kế xây dựng công nghệ thực tế ảo và ứng dụng

NGUYỄN VĂN A

nguyenvanabc@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: PGS. TS. Phạm Văn ABC

Khoa: Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2022

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Thiết kế xây dựng công nghệ thực tế ảo và ứng dụng

NGUYỄN VĂN A

nguyenvanabc@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: PGS. TS. Phạm Văn ABC

Chữ kí GVHD

Khoa:

Kỹ thuật máy tính

Trường:

Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2022

Lời cảm ơn

Đầu tiên, em xin gửi lời tri ân chân thành và sâu sắc nhất đến thầy Trịnh Tuấn Đạt vì đã đưa ra những lời khuyên, những lời hướng dẫn tận tình và đúng đắn cho đến khi em hoàn thành xong ĐATN của mình.

Em cũng xin gửi lời biết ơn sâu sắc tới mái nhà chung Đại học Bách Khoa Hà Nội, đặc biệt là các thầy cô trong chương trình Việt - Nhật. Cảm ơn các thầy cô đã tận tâm giảng dạy, truyền đạt kiến thức và rèn luyện tư duy kỹ thuật cho em trong suốt 4 năm qua. Bách Khoa không chỉ là nơi trao cho em tri thức, mà còn là nơi chấp cánh cho những khát vọng và đam mê tuổi trẻ.

Một lời cảm ơn vô cùng đặc biệt, em muốn dành cho những người bạn đồng hành tuyệt vời của mình. Cảm ơn mọi người vì đã cùng nhau sát cánh qua 4 năm thanh xuân, cùng thức đêm chạy deadline, chia sẻ những áp lực thi cử và tạo nên những kỷ niệm rực rỡ không thể nào quên dưới mái trường này.

Cuối cùng, em xin dành tình yêu thương và lòng biết ơn vô bờ bến đến gia đình hậu phương vững chắc đã luôn thấu hiểu, hy sinh và ủng hộ em vô điều kiện trong suốt chặng đường học tập.

Mặc dù đã nỗ lực hết sức để hoàn thiện đồ án, nhưng do giới hạn về thời gian cũng như kinh nghiệm thực tiễn, đề tài chắc chắn vẫn còn những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu từ quý thầy cô trong Hội đồng đánh giá để em có cơ hội học hỏi và hoàn thiện bản thân hơn nữa.

Em xin chân thành cảm ơn!

Lời cam kết

Họ và tên sinh viên: Nguyễn Ngọc Quân

MSSV: 20225662

Điện thoại liên lạc: 0776254126

Email: quannn225662@sis.hust.edu.vn

Lớp: Việt Nhật 01 - K67

Chương trình đào tạo: Công nghệ thông tin Việt - Nhật 2022

Em – Nguyễn Ngọc Quân – cam kết Đồ án Tốt nghiệp (ĐATN) là công trình nghiên cứu của bản thân em dưới sự hướng dẫn của TS. Trịnh Tuấn Đạt. Các kết quả nêu trong ĐATN là trung thực, là thành quả của riêng em, không sao chép theo bất kỳ công trình nào khác. Tất cả những tham khảo trong ĐATN – bao gồm hình ảnh, bảng biểu, số liệu, và các câu từ trích dẫn – đều được ghi rõ ràng và đầy đủ nguồn gốc trong danh mục tài liệu tham khảo. Em xin hoàn toàn chịu trách nhiệm với dù chỉ một sao chép vi phạm quy chế của nhà trường.

Hà Nội, ngày . . . tháng 05 năm 2026

Tác giả ĐATN

Nguyễn Ngọc Quân

Tóm tắt nội dung đề án

Trong bối cảnh hiện nay, quy trình tuyển dụng truyền thống đang có nhiều bất cập. Các bộ phận nhân sự (HR) thường xuyên bị quá tải khi phải lọc thủ công hàng trăm hồ sơ trong thời gian ngắn. Ở chiều ngược lại, ứng viên lại thiếu các công cụ và môi trường để rèn luyện kỹ năng phỏng vấn sát với thực tế công việc. Bên cạnh đó, việc xây dựng một nền tảng tuyển dụng hiện đại để giải quyết các vấn đề này đòi hỏi hệ thống phải có tính sẵn sàng cao, dễ mở rộng và tự động hóa được quá trình triển khai để tránh các sai sót do thao tác thủ công.

Nhằm giải quyết những bài toán trên, đề án này tiến hành xây dựng và phát triển JobBridge AI - một hệ thống tuyển dụng thông minh dựa trên kiến trúc Microservices. Bên cạnh các tính năng kết nối việc làm cơ bản, hệ thống còn tích hợp Trí tuệ Nhân tạo để cung cấp trợ lý ảo giúp ứng viên luyện tập phỏng vấn theo kịch bản thực tế. Đồng thời, hệ thống cũng hỗ trợ nhà tuyển dụng tự động phân tích, tóm tắt và đánh giá độ phù hợp của CV một cách nhanh chóng.

Đóng góp nổi bật của đề án nằm ở việc áp dụng hiệu quả quy trình DevOps và các tiêu chuẩn vận hành hệ thống phân tán. Toàn bộ các dịch vụ được container hóa và triển khai trên nền tảng đám mây Azure Kubernetes Service (AKS). Quy trình tích hợp và triển khai liên tục (CI/CD) được thiết lập thông qua GitHub Actions và mô hình GitOps với Argo CD, cho phép cập nhật ứng dụng mà không làm gián đoạn dịch vụ (zero-downtime). Cơ sở hạ tầng của hệ thống cũng được tự động hóa hoàn toàn bằng Terraform (Infrastructure as Code) và được theo dõi, giám sát liên tục thông qua Prometheus cùng Grafana.

Kết quả đạt được của đề án là nền tảng JobBridge AI hoạt động ổn định trên môi trường thực tế, đáp ứng tốt yêu cầu về trải nghiệm người dùng. Đề án đã minh chứng cho sự kết hợp hiệu quả giữa phát triển phần mềm hiện đại, ứng dụng AI và vận hành hệ thống đám mây, tạo tiền đề vững chắc để có thể tiếp tục mở rộng quy mô trong tương lai.

Mục lục

Danh sách hình vẽ

Danh sách bảng

ABSTRACT

Mục này khuyến khích sinh viên viết lại mục “Tóm tắt” đề án tốt nghiệp ở trang trước bằng tiếng Anh. Phần này phải có đầy đủ các nội dung như trong phần tóm tắt bằng tiếng Việt. Sinh viên không nhất thiết phải trình bày mục này.

Nhưng nếu lựa chọn trình bày, sinh viên cần đảm bảo câu từ và ngữ pháp chuẩn tắc, nếu không sẽ có tác dụng ngược, gây phản cảm.

MỤC LỤC

DANH MỤC HÌNH VẼ

DANH MỤC BẢNG BIỂU

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong bối cảnh cuộc cách mạng công nghiệp 4.0, thị trường lao động Việt Nam đang chứng kiến sự thay đổi mạnh mẽ với nhu cầu tuyển dụng ngày càng tăng cao. Tuy nhiên, quá trình kết nối giữa nhà tuyển dụng và ứng viên vẫn còn tồn tại nhiều thách thức. Các phương pháp tuyển dụng truyền thống thường tốn nhiều thời gian, chi phí và thiếu hiệu quả trong việc sàng lọc, đánh giá và lựa chọn ứng viên phù hợp.

Về phía ứng viên, đặc biệt là các bạn sinh viên mới ra trường, thường thiếu kinh nghiệm thực tế và kỹ năng trả lời phỏng vấn. Họ gặp khó khăn trong việc chuẩn bị cho các buổi phỏng vấn, không có cơ hội thực hành và nhận phản hồi để cải thiện. Điều này dẫn đến việc bỏ lỡ nhiều cơ hội việc làm tiềm năng do không thể hiện được hết năng lực của bản thân.

Về phía nhà tuyển dụng, quy trình sàng lọc hồ sơ (CV) thường được thực hiện thủ công, tốn nhiều thời gian và nguồn lực. Việc đánh giá hàng trăm, thậm chí hàng nghìn hồ sơ một cách công bằng và khách quan là một bài toán khó, dễ bị ảnh hưởng bởi các yếu tố chủ quan. Điều này không chỉ làm chậm quá trình tuyển dụng mà còn có nguy cơ bỏ sót những ứng viên tài năng.

Nhận thấy những thách thức đó, việc xây dựng một nền tảng ứng dụng công nghệ Trí tuệ nhân tạo (AI) để tạo cầu nối hiệu quả giữa nhà tuyển dụng và người tìm việc là một nhu cầu cấp thiết. Một hệ thống như vậy có thể đồng thời hỗ trợ ứng viên nâng cao kỹ năng phỏng vấn và giúp nhà tuyển dụng tối ưu hóa quy trình sàng lọc hồ sơ, góp phần nâng cao chất lượng và hiệu quả của thị trường lao động.

1.2 Mục tiêu và phạm vi đề tài

Từ những vấn đề đã nêu, đề tài hướng tới mục tiêu chính là xây dựng một nền tảng web có tên "JobBridge AI" nhằm ứng dụng Trí tuệ nhân tạo để giải quyết các bài toán cốt lõi trong quy trình tuyển dụng. Hệ thống sẽ tập trung vào hai nhóm đối tượng người dùng chính với các mục tiêu cụ thể như sau:

- Hỗ trợ người tìm việc luyện tập phỏng vấn với AI:** Mục tiêu là cung cấp một môi trường giả lập buổi phỏng vấn thực tế, nơi người dùng có thể tương tác trực tiếp với một trợ lý AI. Hệ thống sẽ:
 - Đưa ra các câu hỏi phỏng vấn phù hợp với vị trí công việc mà người dùng lựa chọn.

- Ghi nhận và phân tích câu trả lời của người dùng (dưới dạng văn bản).
- Cung cấp phản hồi, đánh giá chi tiết về nội dung, sự rõ ràng và mức độ liên quan trong câu trả lời, từ đó đưa ra gợi ý để cải thiện.

Phạm vi của tính năng này là xây dựng một giao diện hội thoại tương tác và tích hợp mô hình ngôn ngữ lớn để thực hiện các tác vụ trên.

2. Hỗ trợ nhà tuyển dụng sàng lọc và chấm điểm hồ sơ (CV) tự động: Mục tiêu là tự động hóa và tối ưu hóa quy trình sàng lọc hồ sơ ban đầu. Hệ thống sẽ cho phép nhà tuyển dụng:

- Đăng tải các vị trí tuyển dụng cùng với mô tả công việc (Job Description - JD) chi tiết.
- Nhận hồ sơ ứng tuyển từ người tìm việc.
- Sử dụng AI để tự động phân tích, chấm điểm và xếp hạng mức độ phù hợp của từng hồ sơ so với mô tả công việc.

Phạm vi của tính năng này bao gồm việc xây dựng chức năng quản lý tin tuyển dụng, tiếp nhận hồ sơ và phát triển một thuật toán chấm điểm dựa trên AI để xử lý và xếp hạng hồ sơ.

Ngoài ra, đề tài cũng bao gồm việc xây dựng các chức năng phụ trợ cần thiết cho một nền tảng tuyển dụng như quản lý tài khoản người dùng (ứng viên, nhà tuyển dụng), tạo và quản lý hồ sơ cá nhân, và tìm kiếm việc làm.

1.3 Định hướng giải pháp

Để thực hiện các mục tiêu trên, đề tài đề xuất xây dựng hệ thống theo kiến trúc Microservices cho phần backend và Single Page Application (SPA) cho phần frontend, được triển khai trên nền tảng container hóa và điều phối bằng Kubernetes.

• Kiến trúc tổng thể:

- **Backend:** Được phát triển bằng ngôn ngữ Go với Gin Framework, bao gồm các dịch vụ độc lập: Dịch vụ xác thực (Auth), Dịch vụ quản lý việc làm (Jobs), Dịch vụ AI (AI), và một API Gateway để điều phối các yêu cầu từ client.
- **Frontend:** Xây dựng bằng React và TypeScript, tạo ra một giao diện người dùng hiện đại, linh hoạt và đáp ứng nhanh.
- **Cơ sở dữ liệu:** Sử dụng MongoDB làm cơ sở dữ liệu chính, phù hợp với tính chất linh hoạt của dữ liệu hồ sơ và tin tuyển dụng.
- **Triển khai (DevOps):** Toàn bộ hệ thống được container hóa bằng Docker,

triển khai lên Azure Kubernetes Service (AKS). Quy trình CI/CD được tự động hóa bằng GitHub Actions để xây dựng và đẩy image lên Azure Container Registry (ACR), sau đó sử dụng Argo CD cho quy trình GitOps để tự động cập nhật ứng dụng trên Kubernetes.

- **Giải pháp cho các tính năng AI:**

- **Luyện tập phỏng vấn:** Tận dụng sức mạnh của các mô hình ngôn ngữ lớn (LLMs) thông qua API của OpenAI (ví dụ: GPT-3.5, GPT-4). Hệ thống sẽ gửi câu trả lời của người dùng cùng với ngữ cảnh (câu hỏi phỏng vấn, mô tả công việc) đến API để nhận về các phân tích và gợi ý cải thiện. Để duy trì lịch sử và ngữ cảnh hội thoại, một cơ sở dữ liệu vector (ví dụ: ChromaDB) sẽ được sử dụng để lưu trữ và truy xuất các đoạn hội thoại trước đó.
- **Chấm điểm hồ sơ:** Giải pháp bao gồm hai bước chính. Đầu tiên, hệ thống trích xuất nội dung văn bản từ các file CV (PDF, DOCX). Sau đó, nội dung này cùng với mô tả công việc (JD) sẽ được đưa vào một mô hình AI. Mô hình này, cũng dựa trên LLM, sẽ thực hiện vai trò của một chuyên gia nhân sự để đánh giá, cho điểm và cung cấp một bản tóm tắt về mức độ phù hợp của ứng viên.

Việc lựa chọn các công nghệ này nhằm đảm bảo hệ thống có khả năng mở rộng, bảo trì dễ dàng và hiệu năng cao, đồng thời tận dụng được các tiến bộ mới nhất trong lĩnh vực Trí tuệ nhân tạo.

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày tổng quan về các công nghệ và lý thuyết nền tảng được sử dụng trong đề tài, bao gồm kiến trúc Microservices, các công nghệ phía backend và frontend, nền tảng container hóa với Docker và Kubernetes, cũng như các khái niệm về Trí tuệ nhân tạo và mô hình ngôn ngữ lớn.

Chương 3 đi sâu vào phân tích và thiết kế hệ thống. Chương này mô tả chi tiết về kiến trúc hệ thống, thiết kế cơ sở dữ liệu, và thiết kế giao diện người dùng (UI/UX) cho các luồng chức năng chính.

Chương 4 trình bày về quá trình triển khai và hiện thực hóa hệ thống. Nội dung chương bao gồm việc mô tả chi tiết cách xây dựng các microservice, phát triển giao diện người dùng, và đặc biệt là cách tích hợp các mô hình AI để hiện thực hóa các tính năng thông minh của sản phẩm.

Chương 5 tập trung vào việc đánh giá, kiểm thử và triển khai hệ thống. Chương

này trình bày các kịch bản kiểm thử đã thực hiện, kết quả đạt được, và mô tả quy trình triển khai tự động (CI/CD) của ứng dụng lên môi trường đám mây.

Cuối cùng, Chương 6 tổng kết lại các kết quả đã đạt được của đề án, nêu lên những hạn chế còn tồn tại và đề xuất các hướng phát triển, cải tiến trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

CHƯƠNG 3. Cơ sở lý thuyết

3.1 Tổng quan

Chương 1 đã giới thiệu tổng quan về đề tài, xác định các vấn đề cần giải quyết và đề xuất định hướng giải pháp cho hệ thống "JobBridge AI". Chương này sẽ đi sâu vào việc trình bày các cơ sở lý thuyết, công nghệ và kiến trúc nền tảng đã được lựa chọn và áp dụng để xây dựng hệ thống. Việc hiểu rõ các khái niệm này là tiền đề quan trọng để theo dõi các phần phân tích, thiết kế và hiện thực hệ thống trong các chương tiếp theo.

Nội dung chính của chương bao gồm:

- Giới thiệu về kiến trúc Microservices và lý do lựa chọn.
- Trình bày về các công nghệ cốt lõi được sử dụng ở phía Backend (Go, Gin) và Frontend (React, TypeScript).
- Mô tả về công nghệ container hóa với Docker và nền tảng điều phối container với Kubernetes.
- Giới thiệu các khái niệm cơ bản về Trí tuệ nhân tạo, đặc biệt là các Mô hình ngôn ngữ lớn (LLMs) và cách ứng dụng chúng.

3.2 Kiến trúc Microservices

Kiến trúc microservices là một phương pháp phát triển phần mềm, trong đó một ứng dụng lớn được xây dựng dưới dạng một tập hợp các dịch vụ nhỏ, độc lập và có khả năng triển khai riêng lẻ. Mỗi dịch vụ thực thi một quy trình nghiệp vụ cụ thể và giao tiếp với các dịch vụ khác thông qua các giao diện lập trình ứng dụng (API) được định nghĩa rõ ràng, thường là qua giao thức HTTP.

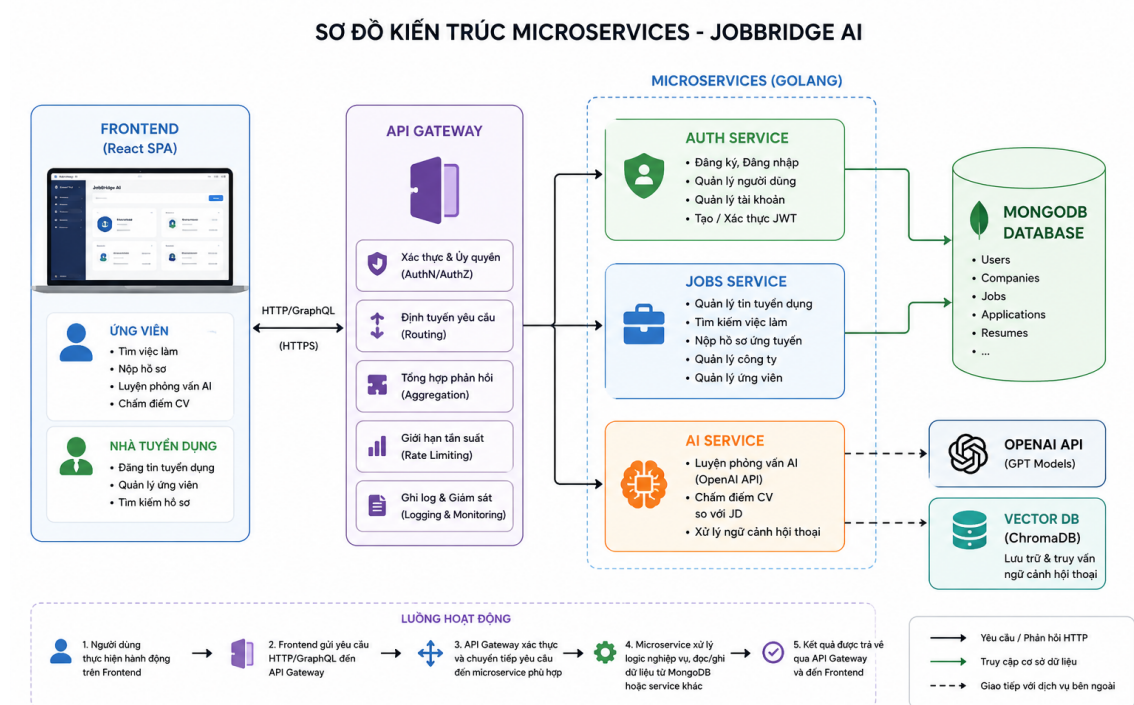
Việc lựa chọn kiến trúc microservices cho dự án "JobBridge AI" xuất phát từ những ưu điểm vượt trội mà nó mang lại so với kiến trúc nguyên khối (monolithic) truyền thống, đặc biệt phù hợp với một hệ thống phức tạp và có định hướng phát triển lâu dài:

- **Tính linh hoạt và khả năng mở rộng:** Mỗi microservice có thể được phát triển, triển khai và mở rộng quy mô một cách độc lập mà không ảnh hưởng đến các thành phần khác của hệ thống. Ví dụ, khi dịch vụ AI cần nhiều tài nguyên xử lý hơn, chúng ta có thể tăng số lượng bản sao (replicas) của chỉ dịch vụ đó.
- **Lựa chọn công nghệ đa dạng:** Mặc dù trong dự án này, tất cả các service đều được viết bằng Go, kiến trúc microservices cho phép mỗi dịch vụ có thể được

xây dựng bằng một công nghệ khác nhau, tùy thuộc vào yêu cầu cụ thể của từng bài toán.

- **Tăng cường khả năng bảo trì và phát triển:** Các dịch vụ nhỏ hơn, tập trung vào một nghiệp vụ duy nhất sẽ dễ hiểu, dễ phát triển và bảo trì hơn. Các nhóm phát triển khác nhau có thể làm việc song song trên các dịch vụ khác nhau, giúp đẩy nhanh tiến độ dự án.
- **Nâng cao khả năng chịu lỗi:** Nếu một dịch vụ gặp sự cố, nó sẽ không làm sập toàn bộ hệ thống. Các dịch vụ khác vẫn có thể tiếp tục hoạt động bình thường.

Như được minh họa trong Hình ??, kiến trúc của "JobBridge AI" bao gồm các thành phần chính sau:



Hình 3.1: Sơ đồ kiến trúc Microservices của JobBridge AI

- **Frontend (React SPA):** Là điểm tương tác duy nhất của người dùng. Tất cả các yêu cầu nghiệp vụ từ phía client sẽ được gửi đến API Gateway.
- **API Gateway:** Đóng vai trò là một lớp trung gian, là cổng vào duy nhất cho hệ thống backend. Nó chịu trách nhiệm xác thực, định tuyến yêu cầu đến các microservice phù hợp, giới hạn tần suất yêu cầu (rate limiting) và tổng hợp dữ liệu trả về cho client.
- **Các Microservices (Go):**
 - **Auth Service:** Chịu trách nhiệm toàn bộ về việc xác thực và quản lý người

dùng (đăng ký, đăng nhập, quản lý tài khoản, cấp phát và xác thực token JWT).

- **Jobs Service:** Xử lý các nghiệp vụ liên quan đến việc làm như quản lý tin tuyển dụng, tìm kiếm, và quy trình ứng tuyển.
- **AI Service:** Là bộ não của hệ thống, thực hiện các tác vụ thông minh như luyện tập phỏng vấn và chấm điểm CV bằng cách tương tác với các dịch vụ bên ngoài như OpenAI API.

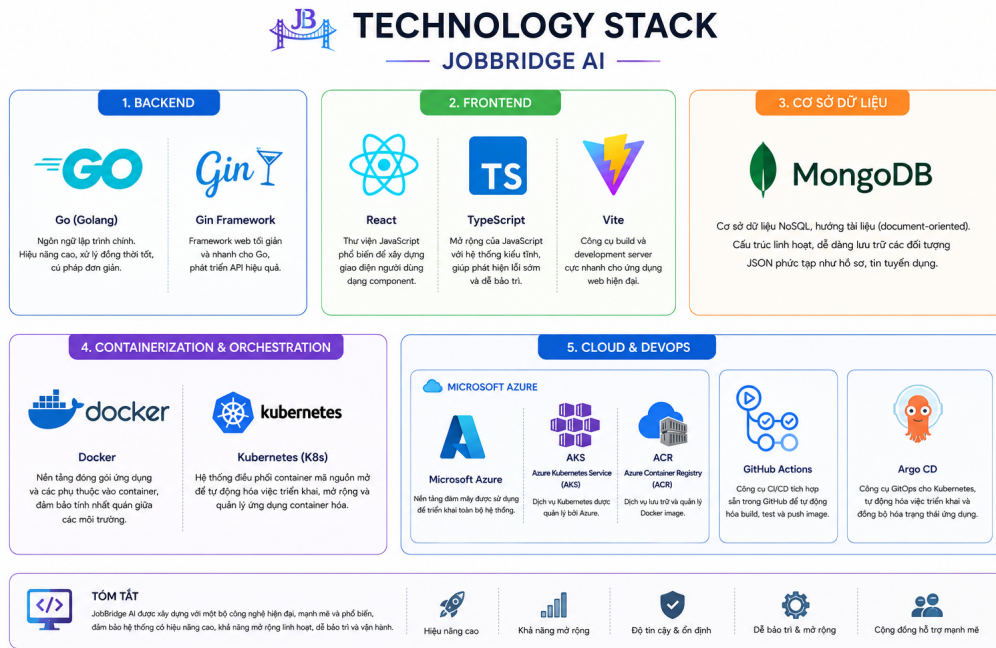
• **Cơ sở dữ liệu:**

- **MongoDB:** Được sử dụng làm cơ sở dữ liệu chính cho Auth Service và Jobs Service để lưu trữ dữ liệu người dùng, công ty, việc làm, và hồ sơ.
- **Vector DB (ChromaDB):** Được đưa vào trong thiết kế kiến trúc mục tiêu để lưu trữ và truy vấn ngữ cảnh hội thoại, giúp tính năng phỏng vấn thử trở nên thông minh và liền mạch hơn.

Luồng hoạt động cơ bản của hệ thống diễn ra theo trình tự: người dùng tương tác trên Frontend, yêu cầu được gửi đến API Gateway, Gateway xác thực và chuyển tiếp đến microservice tương ứng để xử lý, và cuối cùng kết quả được trả về cho người dùng. Mô hình này đảm bảo sự phân tách rõ ràng về trách nhiệm, giúp hệ thống trở nên linh hoạt và dễ dàng quản lý.

3.3 Các công nghệ sử dụng

Việc lựa chọn một bộ công nghệ (technology stack) phù hợp là yếu tố then chốt quyết định đến hiệu năng, khả năng mở rộng và bảo trì của một hệ thống phần mềm. Đối với "JobBridge AI", các công nghệ được lựa chọn đều là những công nghệ hiện đại, có cộng đồng phát triển lớn và đã được chứng minh hiệu quả trong các hệ thống quy mô lớn. Hình ?? cung cấp một cái nhìn tổng quan về các công nghệ được sử dụng trong dự án.



Hình 3.2: Các công nghệ chính được sử dụng trong dự án

3.3.1 Công nghệ phía Backend

Phía backend của hệ thống được xây dựng hoàn toàn bằng ngôn ngữ Go, kết hợp với các công nghệ hỗ trợ để đảm bảo hiệu suất và sự ổn định.

a, Ngôn ngữ Go (Golang)

Go là một ngôn ngữ lập trình biên dịch, mã nguồn mở được phát triển bởi Google. Go được lựa chọn làm ngôn ngữ chính cho việc xây dựng các microservice của "JobBridge AI" vì những lý do sau:

- **Hiệu năng cao:** Là một ngôn ngữ biên dịch, Go cung cấp hiệu suất gần tương đương với các ngôn ngữ như C++ hay Java, rất phù hợp cho các tác vụ xử lý yêu cầu nhanh và hiệu quả.
- **Hỗ trợ đồng thời (Concurrency) mạnh mẽ:** Go có cơ chế hỗ trợ lập trình đồng thời ngay từ trong nhân của ngôn ngữ thông qua Goroutines và Channels. Điều này giúp việc xây dựng các ứng dụng có khả năng xử lý hàng nghìn yêu cầu cùng lúc trở nên đơn giản và hiệu quả hơn rất nhiều so với các cơ chế đa luồng truyền thống.
- **Tối ưu cho ứng dụng mạng và microservice:** Go có một thư viện chuẩn mạnh mẽ, đặc biệt là trong lĩnh vực ứng dụng mạng. Cú pháp đơn giản, thời gian biên dịch nhanh và việc tạo ra một file thực thi duy nhất không có phụ thuộc ngoài giúp cho việc đóng gói và triển khai trong các container trở nên cực kỳ dễ dàng.

b, Gin Web Framework

Gin là một framework phát triển web hiệu suất cao được viết bằng Go. Nó cung cấp các tính năng cần thiết để xây dựng API một cách nhanh chóng nhưng vẫn giữ được hiệu năng tốt. Các đặc điểm nổi bật của Gin bao gồm hệ thống định tuyến (routing) nhanh, khả năng mở rộng thông qua các middleware (ví dụ: logging, xác thực), và khả năng xử lý lỗi tốt.

c, Cơ sở dữ liệu MongoDB

MongoDB là một hệ quản trị cơ sở dữ liệu NoSQL, hướng tài liệu (document-oriented). Thay vì lưu dữ liệu trong các bảng với các hàng và cột như cơ sở dữ liệu quan hệ, MongoDB lưu trữ dữ liệu dưới dạng các tài liệu BSON (một dạng nhị phân của JSON). Lựa chọn này mang lại nhiều lợi ích cho dự án:

- **Cấu trúc linh hoạt (Flexible Schema):** Rất phù hợp để lưu trữ dữ liệu có cấu trúc đa dạng và có thể thay đổi theo thời gian như hồ sơ ứng viên (CVs) hay mô tả công việc (JDs).
- **Khả năng mở rộng ngang:** MongoDB được thiết kế để có thể mở rộng quy mô một cách dễ dàng bằng cách thêm nhiều máy chủ vào cluster.
- **Hiệu năng truy vấn tốt:** Hỗ trợ các chỉ mục (indexes) mạnh mẽ và cung cấp một ngôn ngữ truy vấn phong phú cho các tài liệu JSON.

3.3.2 Công nghệ phía Frontend

Giao diện người dùng của "JobBridge AI" được xây dựng dưới dạng một ứng dụng trang đơn (Single Page Application - SPA) sử dụng các công nghệ hiện đại nhất trong hệ sinh thái JavaScript.

a, React và TypeScript

- **React:** Là một thư viện JavaScript do Facebook phát triển, được sử dụng để xây dựng giao diện người dùng. React cho phép chia nhỏ giao diện thành các thành phần (components) độc lập và có thể tái sử dụng, giúp quản lý trạng thái ứng dụng một cách hiệu quả và làm cho mã nguồn trở nên có tổ chức, dễ bảo trì.
- **TypeScript:** Là một ngôn ngữ lập trình được xây dựng trên nền tảng JavaScript, bổ sung thêm hệ thống kiểu tĩnh (static type system). Việc sử dụng TypeScript giúp phát hiện sớm các lỗi tiềm ẩn ngay trong quá trình phát triển, cải thiện chất lượng mã nguồn và tăng cường khả năng hỗ trợ từ các công cụ lập trình (code completion, refactoring).

b, Vite

Vite là một công cụ xây dựng (build tool) và máy chủ phát triển (dev server) thế hệ mới. So với các công cụ truyền thống như Webpack, Vite cung cấp một trải nghiệm phát triển vượt trội nhờ vào:

- **Khởi động máy chủ cực nhanh:** Tận dụng cơ chế native ES modules của trình duyệt, Vite có thể khởi động máy chủ phát triển gần như ngay lập tức.
- **Hot Module Replacement (HMR) siêu tốc:** Khi có sự thay đổi trong mã nguồn, Vite có thể cập nhật các thay đổi đó trên trình duyệt mà không cần tải lại toàn bộ trang, giúp chu trình phát triển diễn ra nhanh hơn đáng kể.

3.4 Nền tảng triển khai và DevOps

DevOps là một tập hợp các phương pháp luận kết hợp giữa phát triển phần mềm (Development) và vận hành công nghệ thông tin (Operations) nhằm mục đích rút ngắn vòng đời phát triển hệ thống và cung cấp các bản cập nhật, tính năng và sửa lỗi một cách liên tục, đồng thời đảm bảo chất lượng của phần mềm. Dự án "JobBridge AI" áp dụng triệt để các nguyên tắc DevOps thông qua việc sử dụng các công nghệ container hóa, điều phối và tự động hóa quy trình CI/CD.

3.4.1 Container hóa với Docker

Docker là một nền tảng mã nguồn mở cho phép tự động hóa việc triển khai ứng dụng bên trong các "container". Một container là một đơn vị phần mềm nhẹ, độc lập, có thể thực thi, bao gồm mọi thứ cần thiết để chạy một ứng dụng: mã nguồn, runtime, các công cụ hệ thống, và thư viện.

Trong dự án "JobBridge AI", mỗi microservice (Auth, Jobs, AI, Gateway) và cả ứng dụng Frontend đều được đóng gói vào một Docker image riêng biệt thông qua một file 'Dockerfile'. Việc sử dụng Docker mang lại các lợi ích cốt lõi:

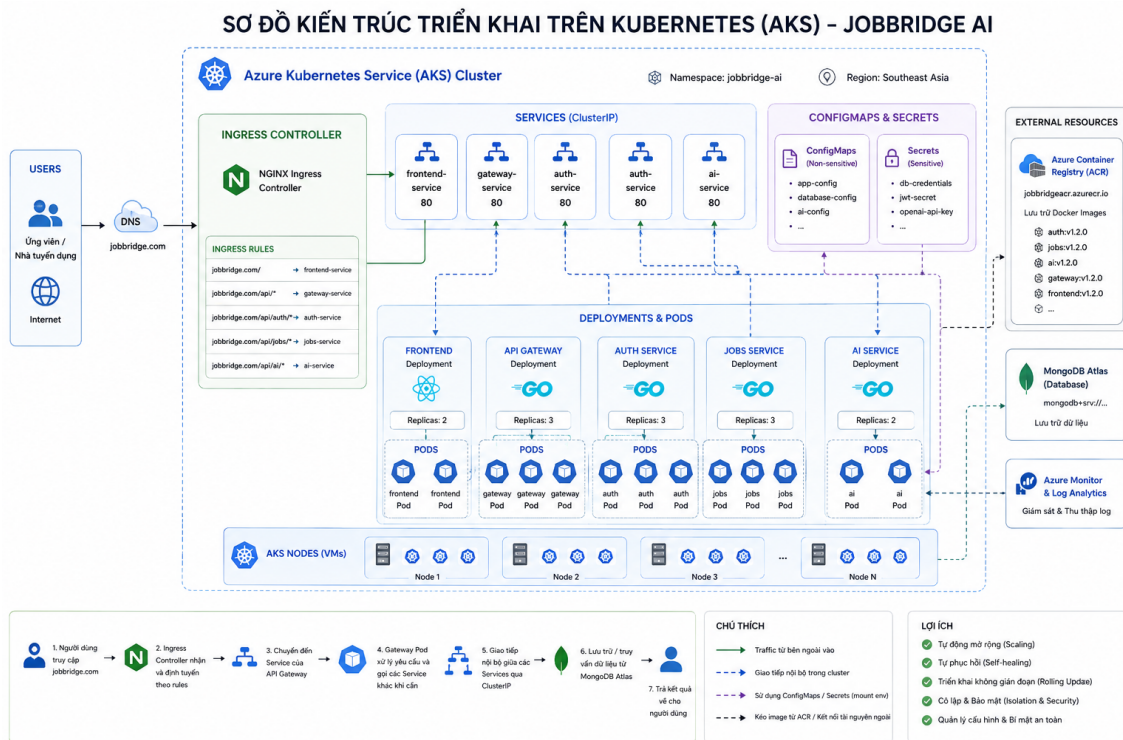
- **Tính nhất quán môi trường:** Đảm bảo rằng ứng dụng sẽ chạy theo cùng một cách, bất kể nó được triển khai ở đâu, từ máy tính cá nhân của lập trình viên cho đến môi trường production trên đám mây. Điều này giúp loại bỏ hoàn toàn vấn đề "It works on my machine".
- **Tính di động:** Docker container có thể chạy trên bất kỳ hệ thống nào có cài đặt Docker, từ Windows, macOS, Linux cho đến các nền tảng đám mây.
- **Tách biệt và bảo mật:** Các container chạy độc lập với nhau và với hệ điều hành chủ, giúp tăng cường tính bảo mật và ổn định.
- **Hiệu quả và nhẹ nhàng:** So với máy ảo (Virtual Machine), container chia sẻ nhân (kernel) của hệ điều hành chủ, giúp chúng khởi động nhanh hơn và tiêu

thụ ít tài nguyên hơn đáng kể.

3.4.2 Điều phối với Kubernetes

Khi một ứng dụng bao gồm nhiều container (như kiến trúc microservices), việc quản lý, kết nối và mở rộng quy mô chúng một cách thủ công sẽ trở nên vô cùng phức tạp. Kubernetes (thường được viết tắt là K8s) là một hệ thống điều phối container mã nguồn mở, được thiết kế để tự động hóa việc triển khai, mở rộng và vận hành các ứng dụng container hóa.

Dự án sử dụng Azure Kubernetes Service (AKS), một dịch vụ Kubernetes được quản lý hoàn toàn bởi Microsoft Azure, giúp đơn giản hóa việc triển khai và quản lý cluster. Các khái niệm chính của Kubernetes được áp dụng trong dự án được mô tả trong Hình ??:



Hình 3.3: Kiến trúc triển khai trên Azure Kubernetes Service

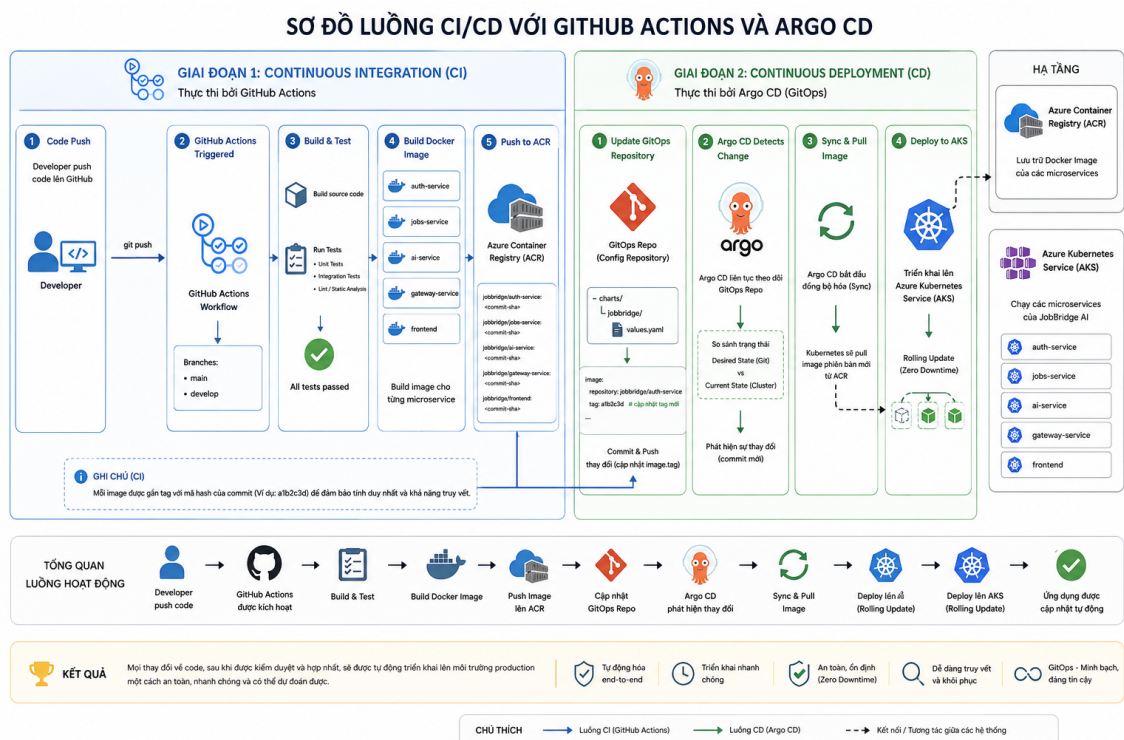
- **Pod:** Là đơn vị nhỏ nhất có thể triển khai trong Kubernetes, chứa một hoặc nhiều container. Trong dự án này, mỗi Pod sẽ chạy một container của một microservice.
- **Deployment:** Là một tài nguyên của Kubernetes dùng để quản lý các Pod. Nó đảm bảo rằng một số lượng bản sao (replicas) nhất định của một Pod luôn ở trạng thái sẵn sàng, đồng thời quản lý quá trình cập nhật phiên bản (rolling update) một cách an toàn.
- **Service:** Cung cấp một điểm truy cập mạng ổn định (một địa chỉ IP và DNS

name cố định) cho một nhóm các Pod. Điều này cho phép các microservice giao tiếp với nhau một cách dễ dàng bên trong cluster.

- **Ingress:** Quản lý truy cập từ bên ngoài vào các Service trong cluster, thường là các yêu cầu HTTP/HTTPS. Nó hoạt động như một bộ định tuyến thông minh, hướng lưu lượng truy cập đến đúng dịch vụ dựa trên tên miền hoặc đường dẫn.

3.4.3 CI/CD với GitHub Actions và Argo CD

Quy trình Tích hợp liên tục (CI) và Triển khai liên tục (CD) là trái tim của thực hành DevOps, giúp tự động hóa toàn bộ quá trình từ việc viết mã cho đến khi sản phẩm đến tay người dùng. Dự án áp dụng một quy trình CI/CD hiện đại dựa trên GitOps, được minh họa trong Hình ??.



Hình 3.4: Sơ đồ luồng CI/CD của dự án

a, Tích hợp liên tục (CI) với GitHub Actions

Quy trình CI được kích hoạt mỗi khi có mã nguồn mới được đẩy lên repository trên GitHub.

1. **Build và Test:** GitHub Actions tự động biên dịch mã nguồn và chạy các bộ kiểm thử (unit test) để đảm bảo chất lượng và tính đúng đắn của các thay đổi.
2. **Build Docker Image:** Nếu tất cả các bài kiểm thử đều thành công, quy trình sẽ tiến hành xây dựng các Docker image cho từng microservice.
3. **Push to Azure Container Registry (ACR):** Các image sau khi được build sẽ

được đẩy lên và lưu trữ tại ACR, một dịch vụ registry container riêng tư trên Azure.

b, Triển khai liên tục (CD) theo mô hình GitOps với Argo CD

GitOps là một phương pháp thực hành CD, trong đó một repository Git (GitOps repo) được coi là "nguồn chân lý" (source of truth) duy nhất cho việc định nghĩa trạng thái mong muốn của hệ thống.

1. **Update GitOps Repo:** Sau khi CI thành công, một bước tự động sẽ cập nhật file cấu hình (ví dụ: file 'values.yaml' của Helm) trong một repository Git thứ hai (GitOps repo), trở đến phiên bản image mới nhất vừa được đẩy lên ACR.
2. **Argo CD:** Là một công cụ GitOps chạy trong cluster Kubernetes. Nó liên tục theo dõi GitOps repo.
3. **Sync and Deploy:** Khi phát hiện có sự thay đổi trong GitOps repo, Argo CD sẽ tự động so sánh trạng thái mong muốn trong repo với trạng thái hiện tại trên cluster. Nếu có sự khác biệt, nó sẽ tự động "đồng bộ" (sync) bằng cách ra lệnh cho Kubernetes kéo image mới về và triển khai phiên bản cập nhật của ứng dụng.

Mô hình này đảm bảo rằng tất cả các thay đổi đối với môi trường vận hành đều được theo dõi, có thể kiểm tra và dễ dàng phục hồi thông qua lịch sử của Git.

3.5 Trí tuệ nhân tạo và Mô hình ngôn ngữ lớn (LLM)

Trí tuệ nhân tạo (Artificial Intelligence - AI) là một lĩnh vực rộng lớn của khoa học máy tính, tập trung vào việc tạo ra các hệ thống thông minh có khả năng thực hiện các tác vụ mà thông thường đòi hỏi trí thông minh của con người. Trong những năm gần đây, một nhánh của AI đã có những bước phát triển đột phá và thu hút sự chú ý đặc biệt, đó là các Mô hình ngôn ngữ lớn (Large Language Models - LLMs).

a, Mô hình ngôn ngữ lớn (LLM)

LLMs là các mô hình AI được huấn luyện trên một khối lượng dữ liệu văn bản khổng lồ, giúp chúng có được khả năng hiểu và tạo ra ngôn ngữ tự nhiên của con người một cách đáng kinh ngạc. Các mô hình này, chẳng hạn như dòng mô hình GPT (Generative Pre-trained Transformer) của OpenAI, có thể thực hiện một loạt các tác vụ ngôn ngữ phức tạp mà không cần được huấn luyện lại một cách cụ thể cho từng tác vụ.

Thay vì phải xây dựng các thuật toán riêng biệt cho mỗi bài toán, chúng ta có thể "ra lệnh" cho LLM thực hiện các yêu cầu thông qua các câu lệnh (prompts). Trong phạm vi của dự án "JobBridge AI", hai khả năng cốt lõi của LLM được tận dụng là:

- **Khả năng hội thoại và nhập vai (Role-playing):** Mô hình có thể đóng vai một nhà tuyển dụng, đưa ra câu hỏi và cung cấp những phản hồi, nhận xét như một chuyên gia thực thụ.
- **Khả năng phân tích, so sánh và tóm tắt nội dung:** Mô hình có thể đọc, hiểu và đối chiếu thông tin từ nhiều nguồn văn bản khác nhau (cụ thể là CV và mô tả công việc) để đưa ra đánh giá và chấm điểm.

Kiến trúc nền tảng của hầu hết các LLM hiện đại là kiến trúc Transformer, cho phép mô hình xử lý các chuỗi dữ liệu (như câu chữ) và nắm bắt được mối quan hệ ngữ cảnh giữa các từ, kể cả khi chúng ở xa nhau trong một đoạn văn.

b, Ứng dụng LLM trong JobBridge AI

Dự án "JobBridge AI" không xây dựng một mô hình LLM từ đầu, mà tận dụng sức mạnh của các mô hình đã được huấn luyện sẵn thông qua các giao diện lập trình ứng dụng (API), cụ thể là OpenAI API. Cách tiếp cận này cho phép dự án tích hợp các khả năng AI tiên tiến một cách nhanh chóng và hiệu quả mà không đòi hỏi nguồn lực tính toán khổng lồ cho việc huấn luyện mô hình.

- **Trong tính năng luyện tập phỏng vấn:** Hệ thống gửi câu hỏi phỏng vấn và câu trả lời của người dùng đến LLM. Với vai trò là một chuyên gia tuyển dụng, LLM sẽ phân tích câu trả lời dựa trên các tiêu chí như sự liên quan, độ rõ ràng, cách diễn đạt và đưa ra những phản hồi mang tính xây dựng, giúp người dùng cải thiện kỹ năng của mình.
- **Trong tính năng chấm điểm CV:** Hệ thống trích xuất nội dung văn bản từ file CV của ứng viên và mô tả công việc (JD) từ nhà tuyển dụng. Cả hai nội dung này sẽ được đưa vào LLM. Mô hình sẽ thực hiện việc so sánh, đối chiếu kinh nghiệm, kỹ năng trong CV với các yêu cầu trong JD để đưa ra một điểm số đánh giá mức độ phù hợp và một bản tóm tắt ngắn gọn về điểm mạnh, điểm yếu của ứng viên.

Bằng cách này, "JobBridge AI" đã biến một công nghệ AI phức tạp thành một công cụ hữu ích và dễ tiếp cận, trực tiếp giải quyết các bài toán thực tế trong quy trình tuyển dụng.

3.6 Kết chương

Chương này đã trình bày một cách có hệ thống các cơ sở lý thuyết và công nghệ nền tảng của dự án "JobBridge AI". Từ kiến trúc Microservices, các công nghệ cụ thể cho backend và frontend, cho đến nền tảng triển khai DevOps hiện đại và các khái niệm về AI, tất cả đều là những mảnh ghép quan trọng tạo nên hệ thống. Những kiến thức này sẽ là cơ sở để đi vào chương tiếp theo, Chương 3, nơi sẽ trình

bày chi tiết về việc phân tích và thiết kế hệ thống dựa trên các nền tảng đã chọn.

CHƯƠNG 4. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

CHƯƠNG 5. Phân tích và Thiết kế hệ thống

5.1 Tổng quan

Chương 2 đã trình bày các cơ sở lý thuyết và công nghệ nền tảng được sử dụng trong dự án. Dựa trên nền tảng đó, chương này sẽ đi sâu vào việc phân tích các yêu cầu của hệ thống "JobBridge AI" và trình bày chi tiết bản thiết kế kỹ thuật. Mục tiêu của chương này là chuyển đổi từ các yêu cầu nghiệp vụ thành một mô hình hệ thống rõ ràng, sẵn sàng cho việc triển khai ở chương sau.

Nội dung chính của chương bao gồm:

- Phân tích các yêu cầu chức năng và phi chức năng của hệ thống.
- Trình bày các sơ đồ ca sử dụng (Use Case) để mô tả tương tác của người dùng.
- Thiết kế chi tiết cho cơ sở dữ liệu.
- Thiết kế giao diện người dùng (UI/UX) cho các luồng nghiệp vụ chính.

5.2 Phân tích yêu cầu

Hệ thống JobBridge AI được xây dựng nhằm giải quyết các thách thức trong quy trình tuyển dụng hiện đại, mang lại lợi ích cho cả ứng viên và nhà tuyển dụng. Để đạt được mục tiêu này, hệ thống cần đáp ứng các yêu cầu chức năng và phi chức năng cụ thể.

5.2.1 Yêu cầu chức năng

Yêu cầu chức năng (Functional Requirements) là những gì hệ thống phải làm. Dựa trên các vai trò người dùng chính là Ứng viên và Nhà tuyển dụng, các yêu cầu chức năng được phân loại như sau:

a, Đối với Ứng viên

- **F1 - Quản lý tài khoản:** Ứng viên có thể đăng ký, đăng nhập, đăng xuất và quản lý thông tin hồ sơ cá nhân.
- **F2 - Tìm kiếm việc làm:** Cung cấp công cụ tìm kiếm và bộ lọc nâng cao (theo vị trí, mức lương, ngành nghề) để ứng viên tìm được công việc phù hợp.
- **F3 - Xem chi tiết việc làm:** Hiển thị đầy đủ thông tin về một tin tuyển dụng, bao gồm mô tả công việc, yêu cầu, và thông tin công ty.
- **F4 - Quản lý CV:** Ứng viên có thể tải lên và lưu trữ nhiều phiên bản CV.
- **F5 - Nộp hồ sơ ứng tuyển:** Cho phép ứng viên nộp hồ sơ (apply) vào các vị trí mong muốn chỉ với vài cú nhấp chuột.

- **F6 - Sử dụng tính năng AI tư vấn phỏng vấn:** Ứng viên có thể tham gia một buổi phỏng vấn giả lập với AI, nhận câu hỏi và được đánh giá câu trả lời.
- **F7 - Nhận gợi ý việc làm từ AI:** Dựa trên nội dung CV và lịch sử tìm kiếm, AI sẽ tự động gợi ý các công việc phù hợp.

b, Đối với Nhà tuyển dụng

- **F9 - Quản lý tài khoản:** Nhà tuyển dụng có thể đăng ký, đăng nhập, và quản lý thông tin công ty.
- **F10 - Đăng tin tuyển dụng:** Cung cấp biểu mẫu chi tiết để nhà tuyển dụng có thể tạo và đăng các vị trí đang cần tuyển.
- **F11 - Quản lý tin tuyển dụng:** Cho phép chỉnh sửa, cập nhật trạng thái (đang mở, đã đóng) hoặc xóa các tin đã đăng.
- **F12 - Quản lý ứng viên:** Xem danh sách các ứng viên đã nộp hồ sơ cho một vị trí cụ thể, xem chi tiết hồ sơ và CV của họ.
- **F13 - Phân tích CV ứng viên bằng AI:** Khi xem một CV, hệ thống tự động dùng AI để tóm tắt, trích xuất các thông tin quan trọng như kỹ năng, kinh nghiệm, trình độ học vấn, giúp nhà tuyển dụng sàng lọc nhanh hơn.

5.2.2 Yêu cầu phi chức năng

Yêu cầu phi chức năng (Non-functional Requirements) định nghĩa các tiêu chuẩn về chất lượng và cách thức hoạt động của hệ thống.

- **NF1 - Hiệu năng (Performance):**
 - Thời gian phản hồi của trang web phải dưới 2 giây cho các tác vụ thông thường.
 - Thời gian phản hồi của AI (tạo CV, trả lời phỏng vấn) phải dưới 5 giây để đảm bảo trải nghiệm tương tác tự nhiên.
 - Hệ thống có khả năng chịu tải ít nhất 100 người dùng đồng thời.
- **NF2 - Tính sẵn sàng (Availability):** Hệ thống phải hoạt động 99.5% thời gian, đảm bảo người dùng có thể truy cập bất cứ lúc nào.
- **NF3 - Khả năng mở rộng (Scalability):** Kiến trúc microservices và triển khai trên Kubernetes cho phép hệ thống dễ dàng nâng cấp (scale-out) từng dịch vụ độc lập khi lưu lượng truy cập tăng cao.
- **NF4 - Bảo mật (Security):**
 - Mật khẩu người dùng phải được băm (hashing) trước khi lưu vào cơ sở dữ

liệu.

- Sử dụng JWT (JSON Web Tokens) để xác thực và phân quyền truy cập API.
- Áp dụng các biện pháp chống lại các tấn công phổ biến như XSS, CSRF.
- Toàn bộ giao tiếp giữa client và server phải được mã hóa bằng HTTPS.
- **NF5 - Tính dễ sử dụng (Usability):** Giao diện người dùng phải trực quan, thân thiện và dễ dàng điều hướng cho cả người dùng không chuyên về công nghệ.
- **NF6 - Khả năng bảo trì (Maintainability):** Mã nguồn được chia thành các module (microservices) rõ ràng, tuân thủ các nguyên tắc code sạch (clean code), có tài liệu và comment đầy đủ để dễ dàng bảo trì và phát triển trong tương lai.

5.3 Thiết kế hệ thống

5.3.1 Thiết kế ca sử dụng (Use Case)

Sơ đồ Use Case được sử dụng để mô hình hóa các yêu cầu chức năng của hệ thống, xác định các tương tác giữa người dùng (actors) và hệ thống.

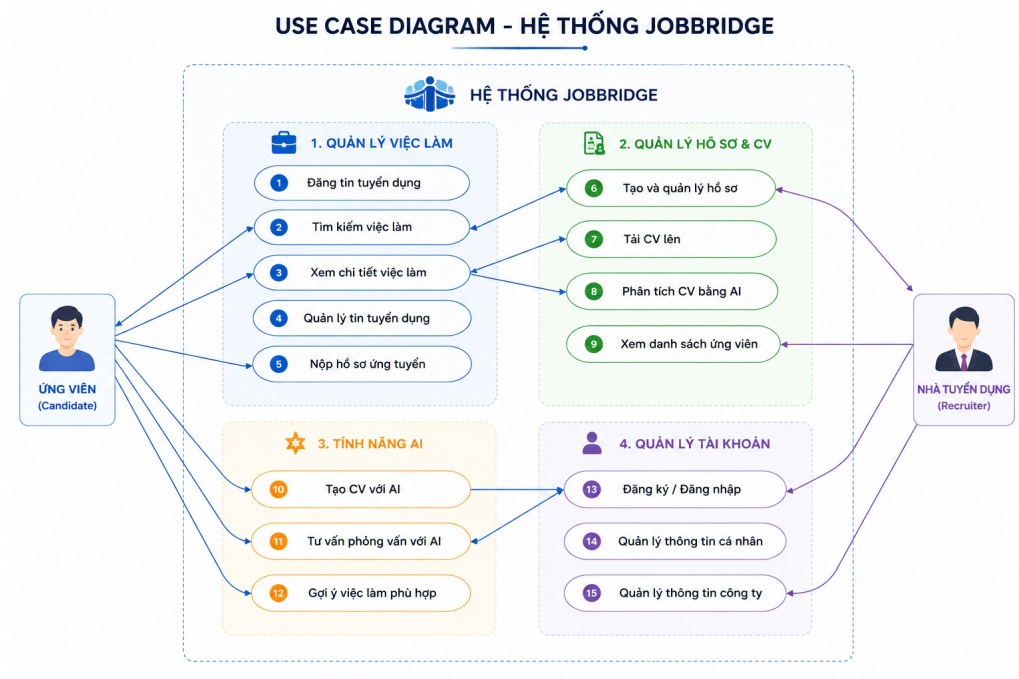
a, Xác định các tác nhân (Actors)

Hệ thống có hai tác nhân chính:

- **Ứng viên (Candidate):** Người tìm việc, sử dụng hệ thống để tìm kiếm công việc, tạo và quản lý hồ sơ, và tận dụng các công cụ AI.
- **Nhà tuyển dụng (Recruiter):** Người đại diện cho công ty, sử dụng hệ thống để đăng tin tuyển dụng và tìm kiếm ứng viên tiềm năng.

b, Sơ đồ Use Case tổng quát

Sơ đồ dưới đây thể hiện một cách tổng quan các chức năng chính mà mỗi tác nhân có thể thực hiện trong hệ thống JobBridge.

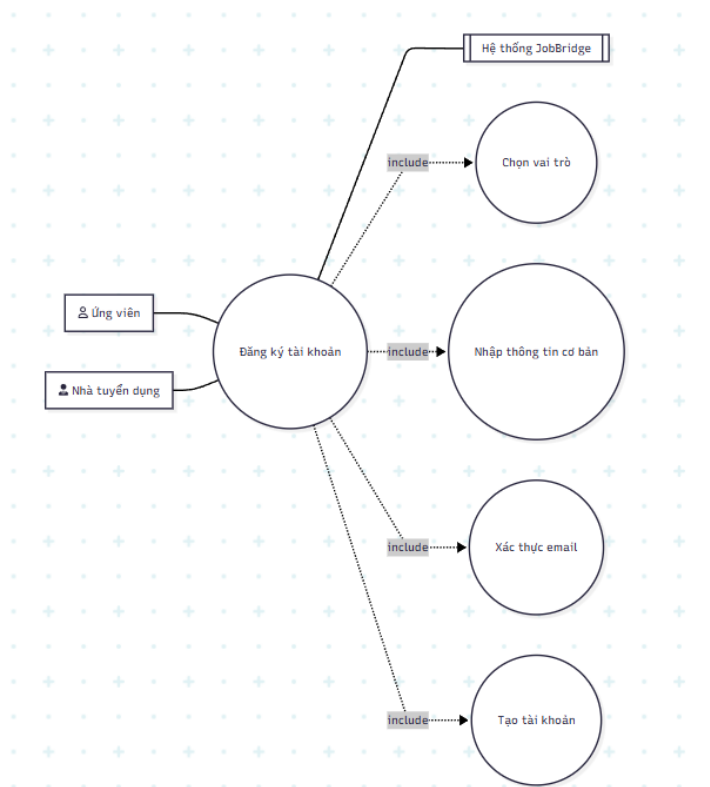


Hình 5.1: Sơ đồ Use Case tổng quát của hệ thống.

c, Sơ đồ Use Case chi tiết

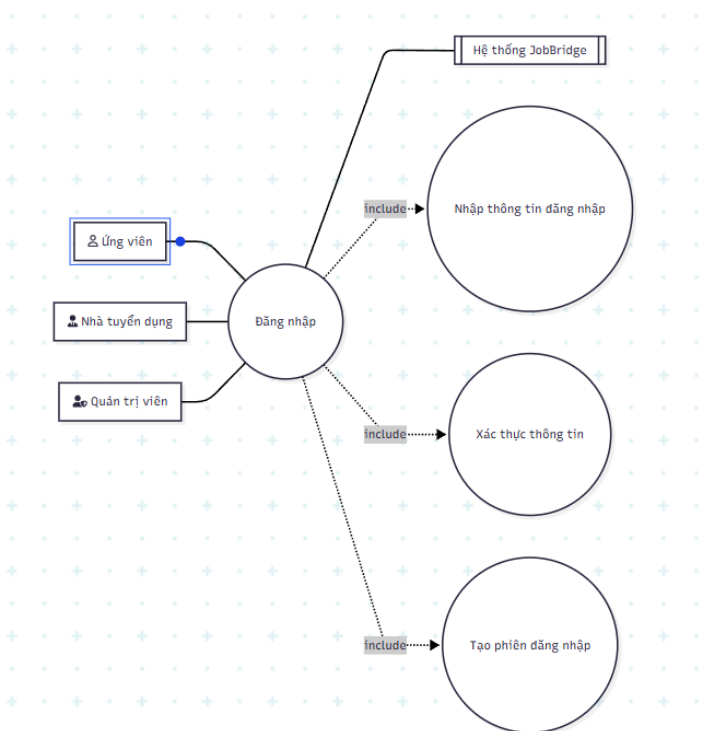
Phần này trình bày các sơ đồ Use Case riêng cho từng chức năng chính của hệ thống. Mỗi sơ đồ thể hiện tác nhân liên quan và các bước con (include) trong luồng nghiệp vụ.

Đăng ký tài khoản. Người dùng mới (ứng viên hoặc nhà tuyển dụng) thực hiện đăng ký bằng cách chọn vai trò, nhập thông tin cơ bản, xác thực email và tạo tài khoản.



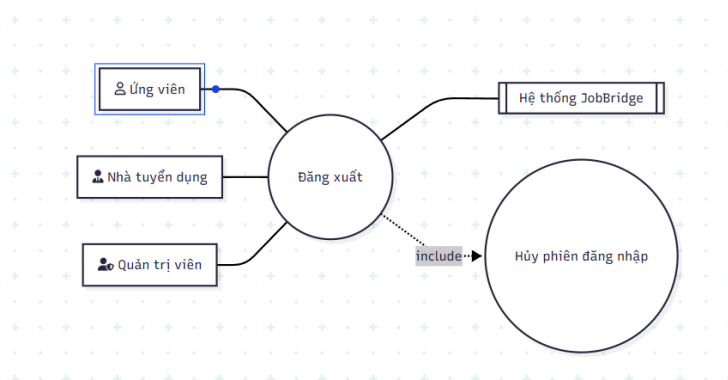
Hình 5.2: Use Case: Đăng ký tài khoản.

Đăng nhập. Người dùng nhập thông tin đăng nhập, hệ thống xác thực và tạo phiên làm việc để truy cập các chức năng tương ứng.



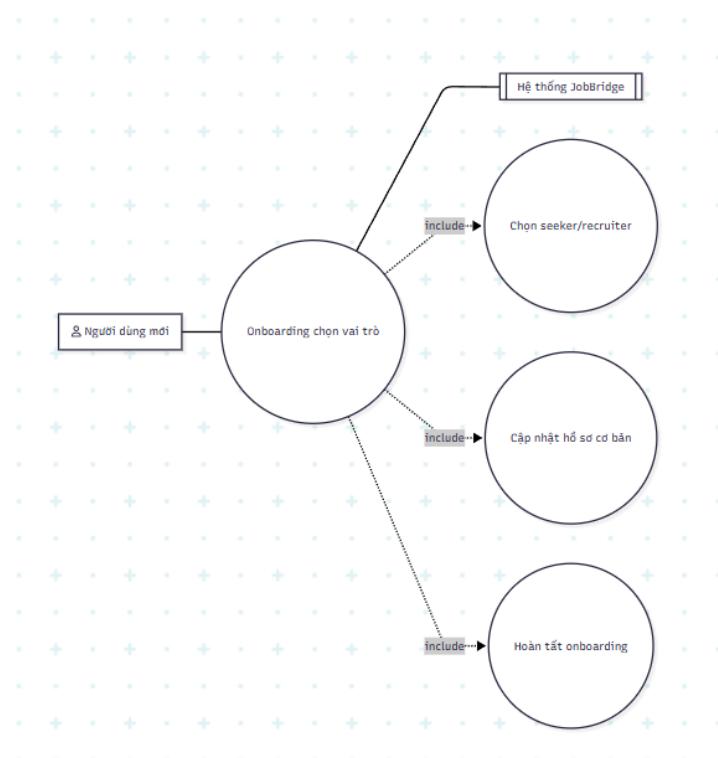
Hình 5.3: Use Case: Đăng nhập.

Đăng xuất. Người dùng kết thúc phiên làm việc; hệ thống hủy phiên đăng nhập để đảm bảo an toàn thông tin.



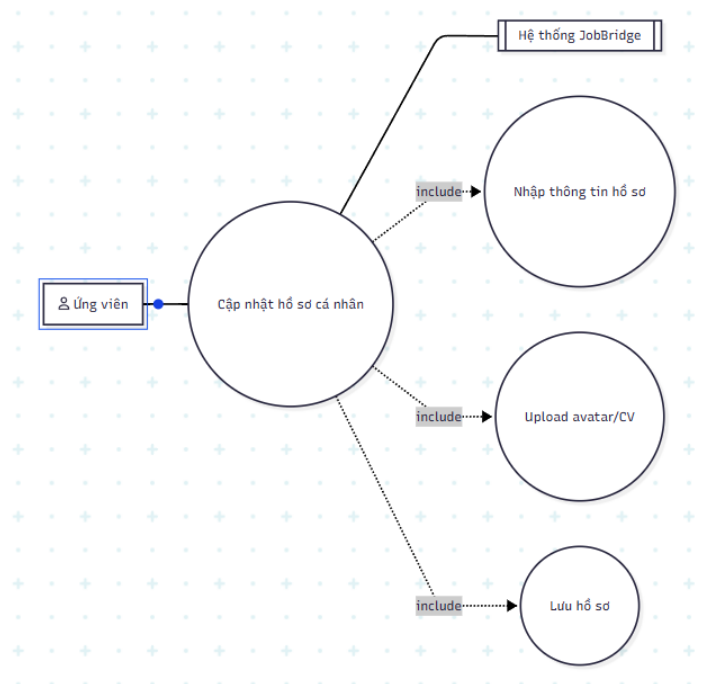
Hình 5.4: Use Case: Đăng xuất.

Onboarding chọn vai trò. Sau khi tạo tài khoản, người dùng chọn vai trò (ứng viên/nhà tuyển dụng) và hoàn tất thông tin cơ bản để kích hoạt trải nghiệm phù hợp.



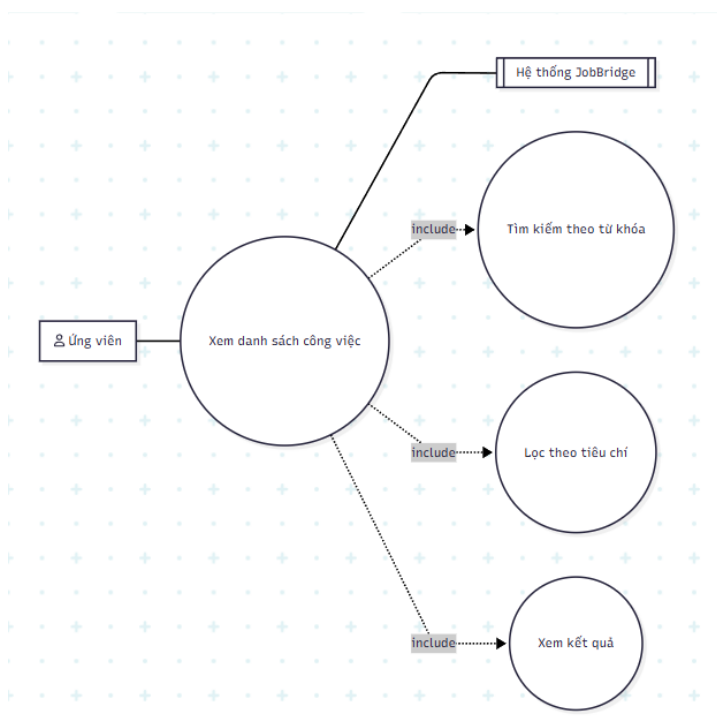
Hình 5.5: Use Case: Onboarding chọn vai trò.

Cập nhật hồ sơ cá nhân. Ứng viên cập nhật thông tin cá nhân và tải lên avatar/CV để hoàn thiện hồ sơ trước khi ứng tuyển.



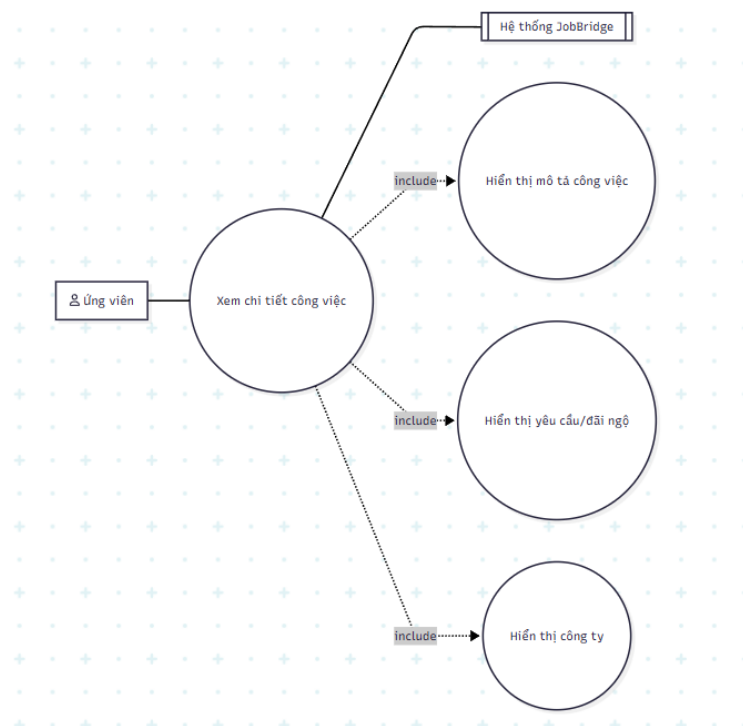
Hình 5.6: Use Case: Cập nhật hồ sơ cá nhân.

Xem danh sách công việc. Ứng viên tìm kiếm, lọc theo tiêu chí và xem danh sách kết quả việc làm phù hợp.



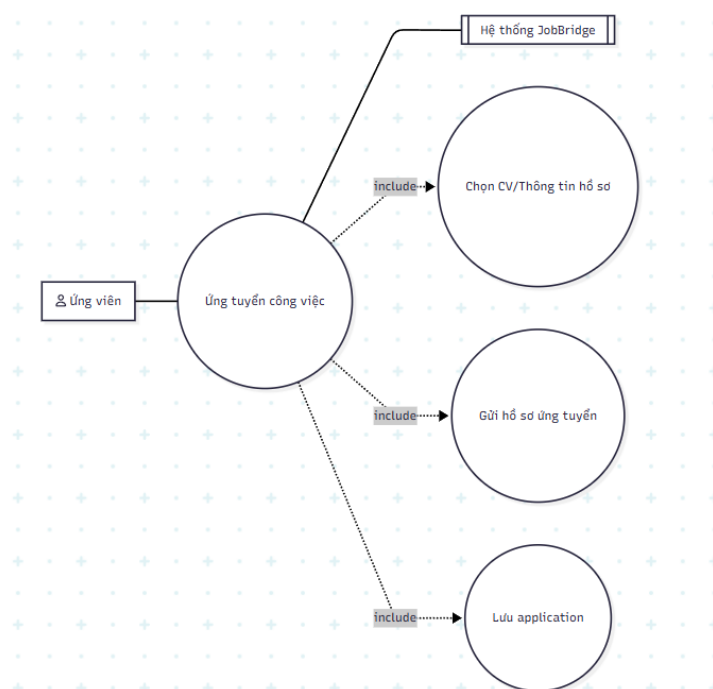
Hình 5.7: Use Case: Xem danh sách công việc.

Xem chi tiết công việc. Ứng viên xem mô tả, yêu cầu và thông tin công ty để quyết định ứng tuyển.



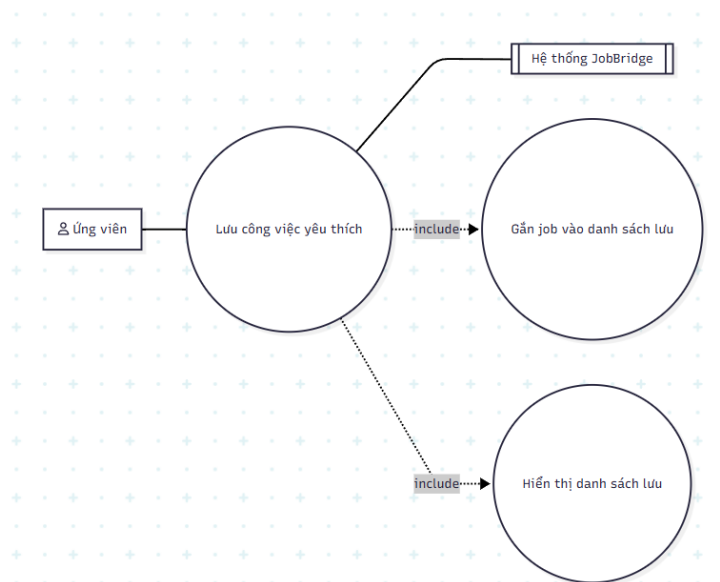
Hình 5.8: Use Case: Xem chi tiết công việc.

Ứng tuyển công việc. Ứng viên chọn CV/hồ sơ phù hợp, gửi hồ sơ ứng tuyển và hệ thống lưu lại thông tin application.



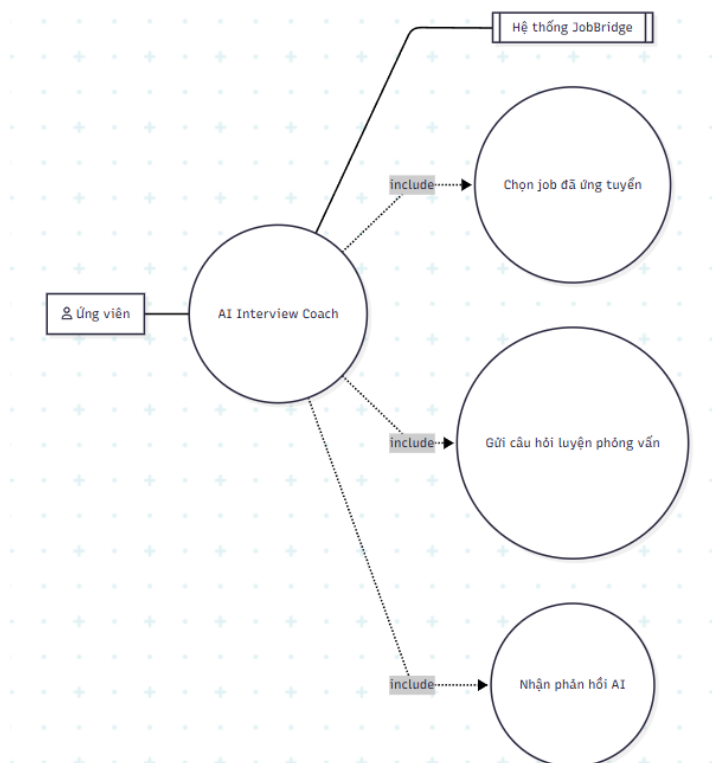
Hình 5.9: Use Case: Ứng tuyển công việc.

Lưu công việc yêu thích. Ứng viên lưu lại các công việc quan tâm để xem và ứng tuyển sau.



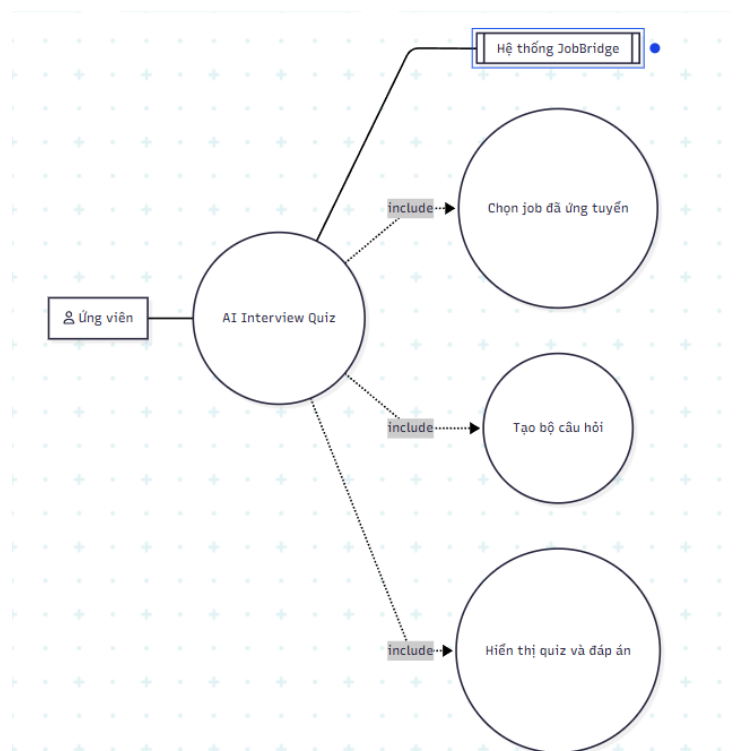
Hình 5.10: Use Case: Lưu công việc yêu thích.

AI Interview Coach. Ứng viên chọn job đã ứng tuyển, hệ thống tạo kịch bản phỏng vấn và phản hồi theo từng lượt trao đổi.



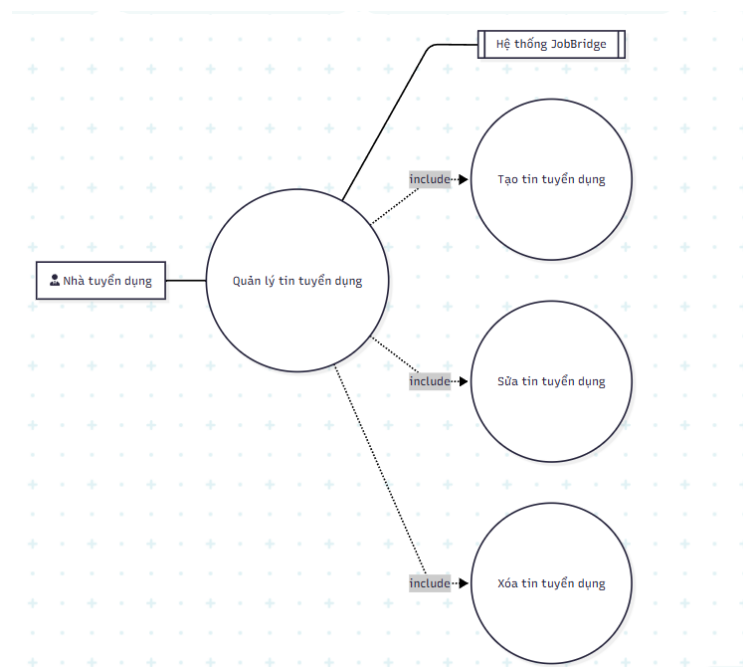
Hình 5.11: Use Case: AI Interview Coach.

AI Interview Quiz. Hệ thống tạo bộ câu hỏi trắc nghiệm theo JD, hiển thị quiz và ghi nhận đáp án của ứng viên.



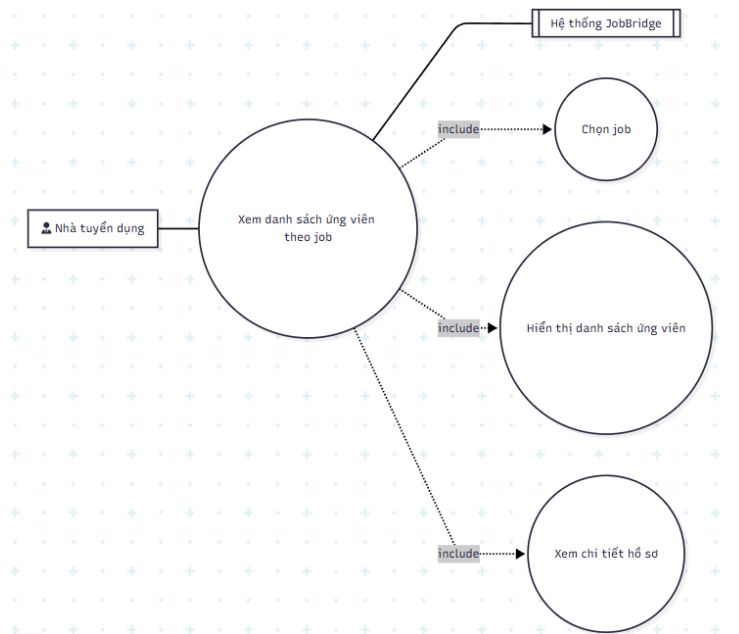
Hình 5.12: Use Case: AI Interview Quiz.

Quản lý tin tuyển dụng. Nhà tuyển dụng tạo, chỉnh sửa và xóa các tin tuyển dụng, cập nhật trạng thái hiển thị.



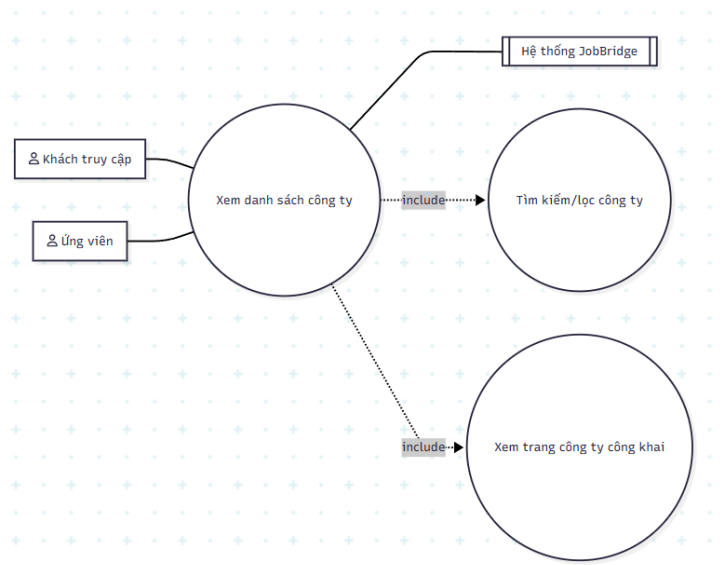
Hình 5.13: Use Case: Quản lý tin tuyển dụng.

Danh sách ứng viên theo job. Nhà tuyển dụng chọn một tin tuyển dụng và xem danh sách ứng viên đã nộp hồ sơ.



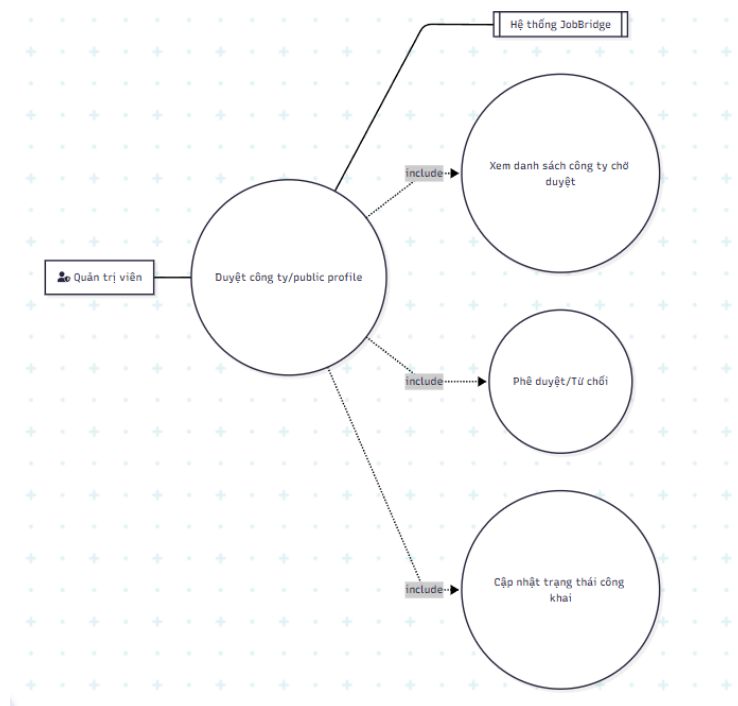
Hình 5.14: Use Case: Xem danh sách ứng viên theo job.

Xem danh sách công ty. Ứng viên/khách truy cập duyệt danh sách công ty, tìm kiếm và lọc theo tiêu chí.



Hình 5.15: Use Case: Xem danh sách công ty.

Duyệt công ty/public profile. Quản trị viên duyệt thông tin công ty, cập nhật trạng thái công khai để hiển thị với người dùng.



Hình 5.16: Use Case: Duyệt công ty/public profile.

d, Mô tả chi tiết một số Use Case quan trọng

• UC5: Nộp hồ sơ ứng tuyển

– **Tác nhân:** Ứng viên.

– **Mô tả:** Ứng viên sau khi tìm thấy một công việc phù hợp có thể tiến hành nộp hồ sơ trực tuyến.

– **Luồng sự kiện:**

1. Ứng viên nhấn nút "Nộp hồ sơ" tại trang chi tiết việc làm.
2. Hệ thống hiển thị form ứng tuyển, yêu cầu chọn CV từ danh sách đã tải lên.
3. Ứng viên chọn CV và xác nhận nộp.
4. Hệ thống ghi nhận thông tin ứng tuyển vào cơ sở dữ liệu và thông báo thành công cho ứng viên.

• UC13: Phân tích CV ứng viên bằng AI

– **Tác nhân:** Nhà tuyển dụng.

– **Mô tả:** Khi nhà tuyển dụng xem CV của một ứng viên, AI sẽ tự động phân tích và đưa ra các thông tin tóm tắt quan trọng.

– **Luồng sự kiện:**

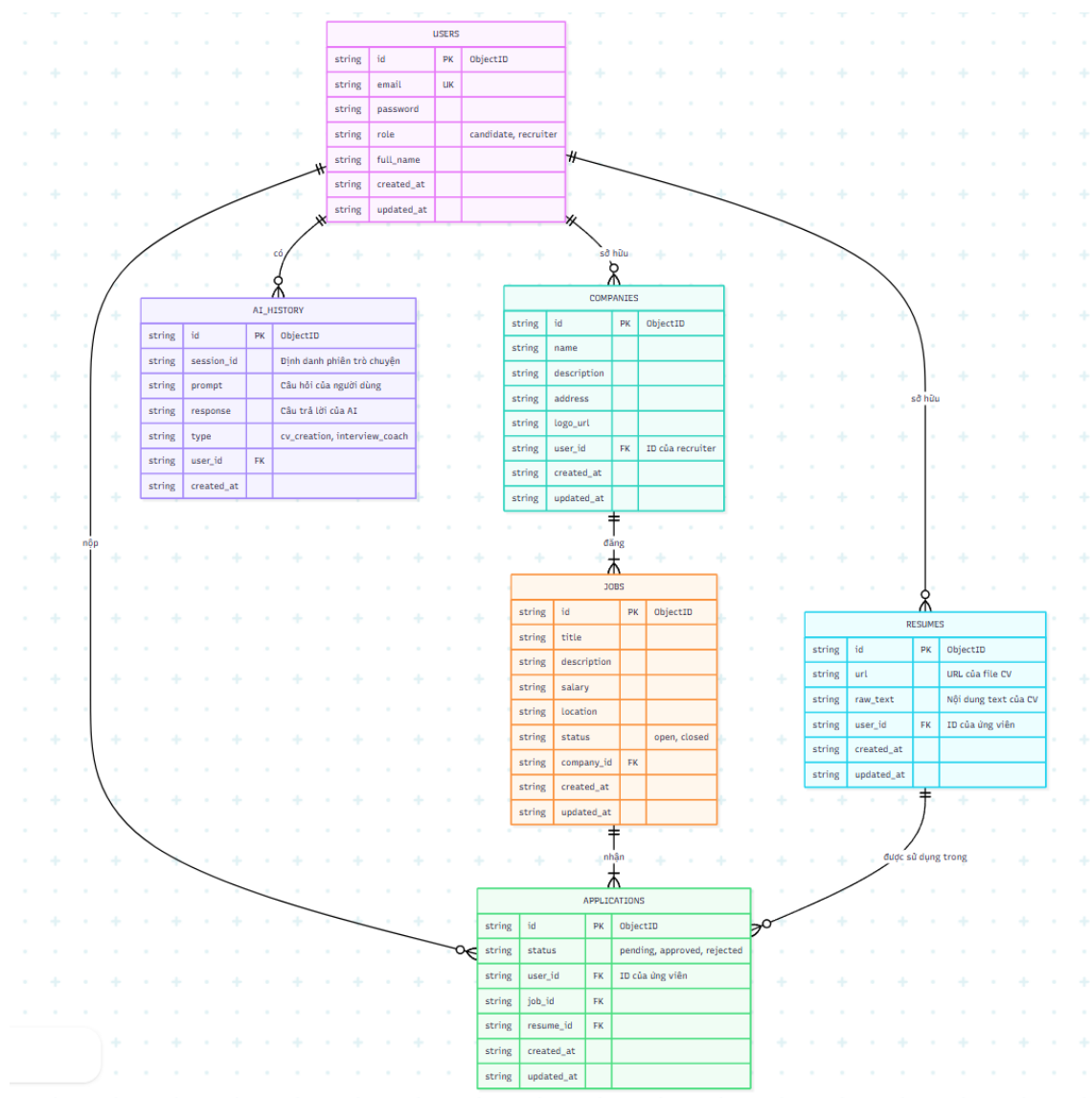
1. Nhà tuyển dụng mở trang chi tiết một hồ sơ ứng tuyển.
2. Hệ thống lấy nội dung text từ file CV và gửi đến service AI.
3. Service AI xử lý văn bản, trích xuất các thực thể như: Kỹ năng (Skills), Kinh nghiệm làm việc (Work Experience), Trình độ học vấn (Education), Thông tin liên hệ (Contact Info).
4. Kết quả phân tích được hiển thị ở một khu vực riêng biệt bên cạnh CV, giúp nhà tuyển dụng nắm bắt thông tin nhanh chóng.

5.3.2 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng MongoDB, một cơ sở dữ liệu NoSQL hướng tài liệu (document-oriented), vì những lý do sau:

- **Linh hoạt:** Cấu trúc schema linh hoạt cho phép dễ dàng lưu trữ các loại dữ liệu đa dạng và thay đổi cấu trúc khi cần, rất phù hợp với các thông tin như hồ sơ người dùng, CV, hay mô tả công việc.
- **Khả năng mở rộng:** MongoDB hỗ trợ mở rộng theo chiều ngang (horizontal scaling) một cách hiệu quả, sẵn sàng cho việc tăng trưởng dữ liệu và lưu lượng truy cập trong tương lai.
- **Hiệu năng:** Khả năng truy vấn và ghi dữ liệu nhanh chóng đối với các tập dữ liệu lớn.

Sơ đồ quan hệ thực thể (ERD) dưới đây mô tả các collection chính và mối quan hệ logic giữa chúng. Cần lưu ý rằng các "khóa ngoại" trong sơ đồ này thể hiện sự tham chiếu ở cấp độ ứng dụng, không phải là ràng buộc được thực thi bởi MongoDB.



Hình 5.17: Sơ đồ quan hệ thực thể (ERD) của hệ thống.

a, Mô tả các collection

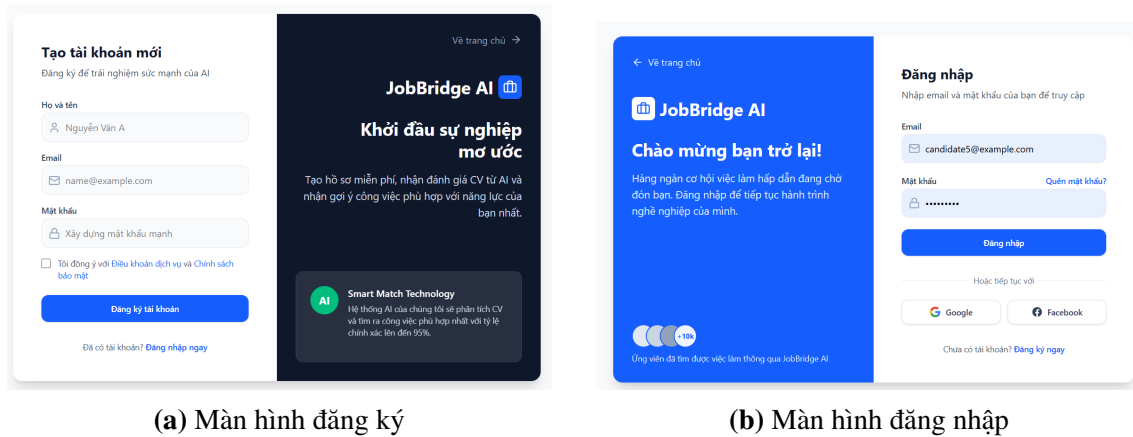
- **users:** Lưu trữ thông tin tài khoản của cả ứng viên và nhà tuyển dụng.
- **companies:** Lưu trữ thông tin chi tiết về các công ty tuyển dụng. Mỗi công ty được quản lý bởi một người dùng có vai trò là 'recruiter'.
- **jobs:** Chứa thông tin về các tin tuyển dụng do các công ty đăng tải.
- **resumes:** Lưu trữ đường dẫn và nội dung text đã được trích xuất từ file CV của ứng viên.
- **applications:** Là collection trung gian, ghi lại lịch sử ứng tuyển của ứng viên vào các công việc.
- **ai_history:** Lưu lại lịch sử các cuộc hội thoại giữa người dùng và các tính năng AI.

5.3.3 Thiết kế giao diện (UI/UX)

Giao diện người dùng được thiết kế theo phong cách hiện đại, tối giản và tập trung vào trải nghiệm người dùng. Dưới đây là thiết kế cho một số màn hình chức năng chính.

a, Xác thực người dùng

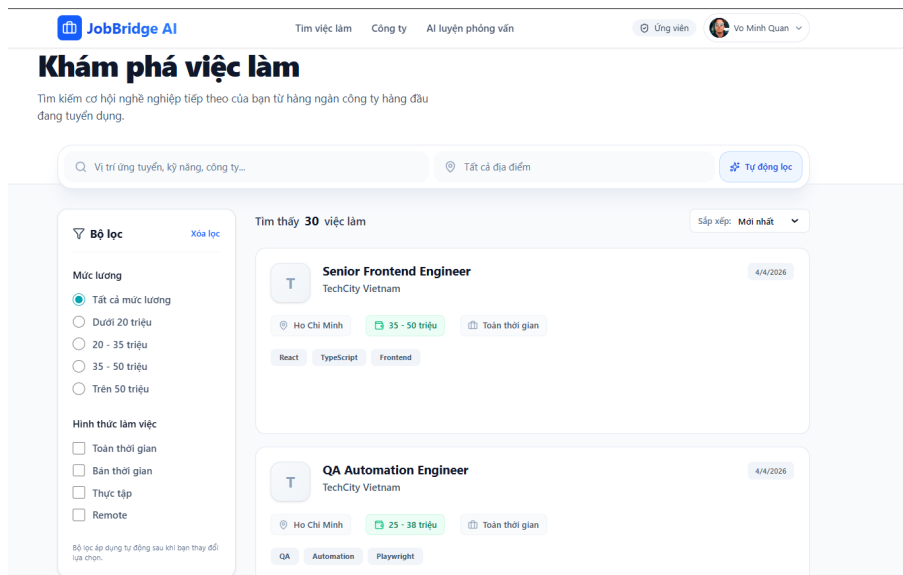
Hệ thống cung cấp các màn hình cho việc đăng ký và đăng nhập.



Hình 5.18: Giao diện các chức năng xác thực.

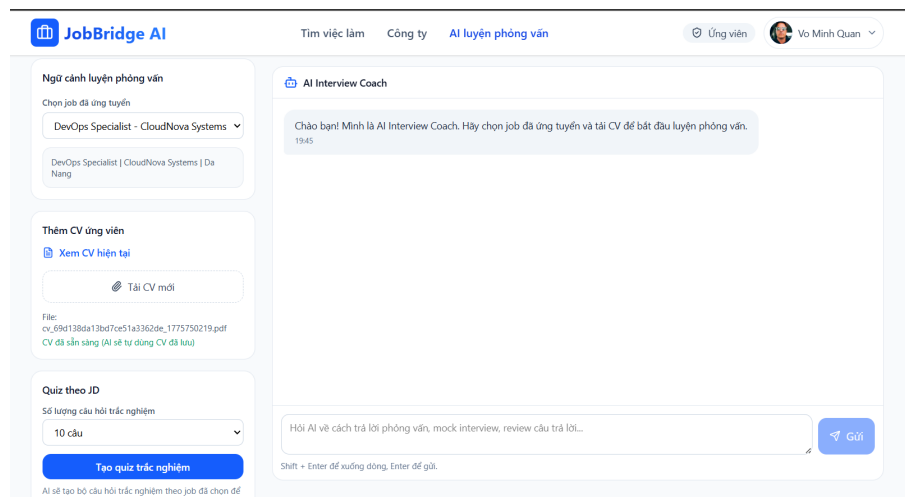
b, Chức năng cho Ứng viên

- **Tìm kiếm và xem việc làm:** Giao diện hiển thị danh sách các công việc, đi kèm bộ lọc chi tiết và trang xem thông tin đầy đủ của công việc.



Hình 5.19: Giao diện trang danh sách việc làm.

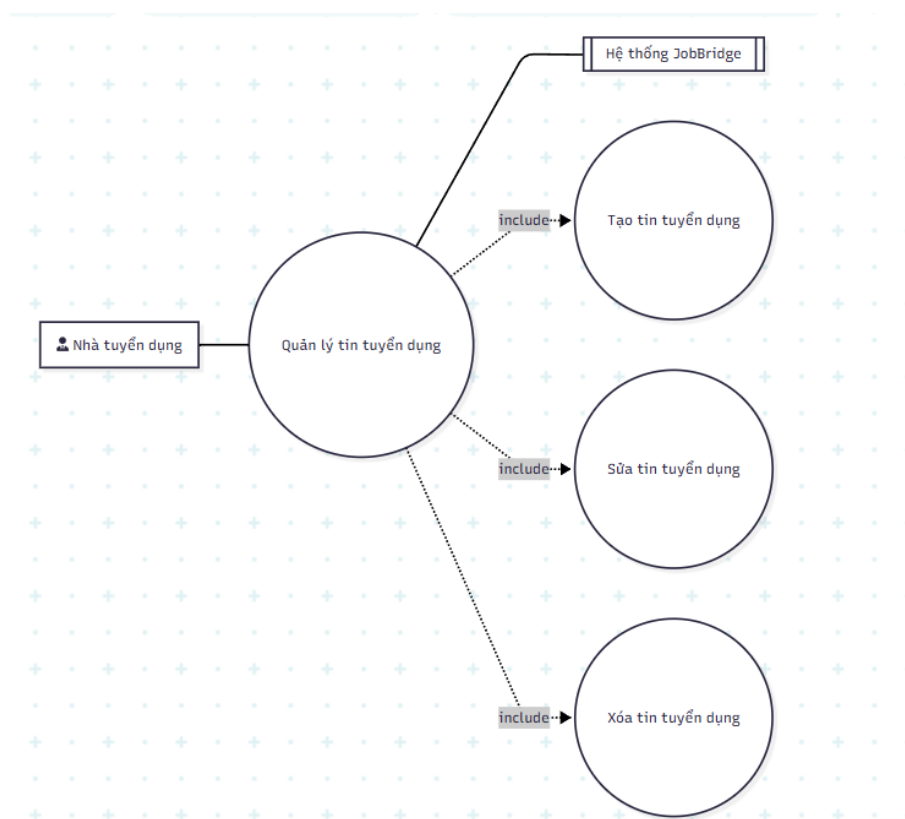
- **Tư vấn phỏng vấn với AI (AI Coach):** Giao diện phỏng vấn giả lập, AI đưa ra câu hỏi và người dùng trả lời.



Hình 5.20: Giao diện chức năng tư vấn phỏng vấn với AI.

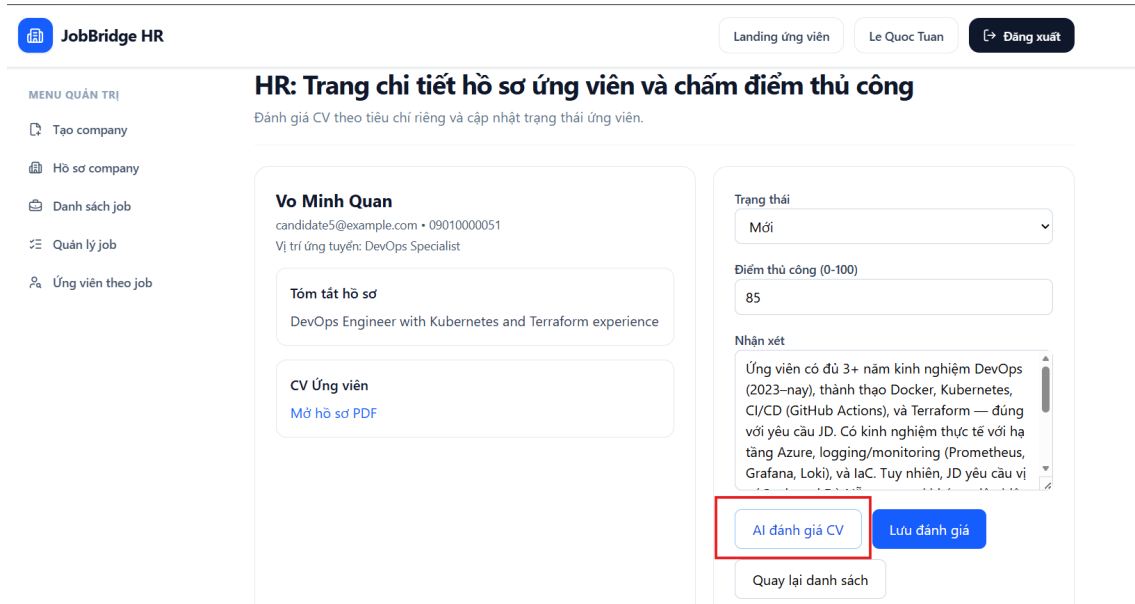
c, Chức năng cho Nhà tuyển dụng

- **Quản lý tin tuyển dụng:** Giao diện để cập nhật trạng thái, chỉnh sửa vị trí cần tuyển.



Hình 5.21: Giao diện trang quản lý tin tuyển dụng.

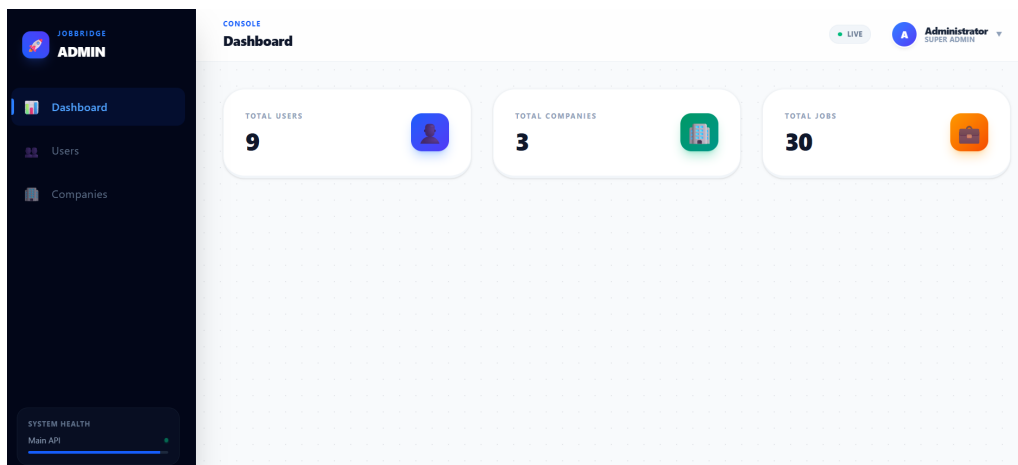
- **Quản lý ứng viên và phân tích CV:** Giao diện hiển thị danh sách ứng viên đã nộp hồ sơ. Khi xem chi tiết, CV của ứng viên được phân tích bởi AI (đánh giá mức độ phù hợp, kỹ năng).



Hình 5.22: Giao diện quản lý ứng viên với tính năng phân tích CV của AI.

d, Chức năng cho Quản trị viên

- **Dashboard quản trị:** Hiển thị tổng quan số lượng người dùng, công ty và tin tuyển dụng, hỗ trợ quản trị viên giám sát hoạt động hệ thống.



Hình 5.23: Giao diện dashboard dành cho quản trị viên.

5.3.4 Thiết kế triển khai và DevOps

Phần này trình bày thiết kế triển khai hệ thống theo hướng DevOps, tập trung vào cách đóng gói, triển khai và tự động hóa quá trình phát hành. Mục tiêu là đảm bảo hệ thống có thể triển khai nhanh, nhất quán và dễ mở rộng.

a, Đóng gói dịch vụ bằng Docker

Mỗi microservice (Auth, Jobs, AI, Gateway) và Frontend được đóng gói thành một Docker image riêng. Thiết kế này đảm bảo tính nhất quán môi trường và cho phép triển khai độc lập từng thành phần. Các image được chuẩn hóa theo cấu trúc

dự án để phục vụ build tự động trong pipeline.

b, Triển khai trên Kubernetes (AKS)

Hệ thống được triển khai trên Azure Kubernetes Service (AKS). Mỗi service chạy trong một Deployment riêng, cấu hình Service nội bộ để giao tiếp và Ingress cho truy cập từ bên ngoài. Thiết kế này hỗ trợ scale-out theo từng service, giúp tối ưu tài nguyên và khả năng chịu tải.

c, Quy trình CI/CD theo mô hình GitOps

Quy trình CI/CD được thiết kế theo GitOps: mã nguồn khi được đẩy lên sẽ kích hoạt GitHub Actions để build, test và đẩy image lên Azure Container Registry (ACR). Sau đó, Argo CD đồng bộ cấu hình triển khai từ repository GitOps, tự động cập nhật các phiên bản mới lên cluster. Cách tiếp cận này giúp kiểm soát tốt lịch sử triển khai và dễ rollback khi cần.

5.4 Kết chương

Trong chương này, hệ thống được phân tích từ yêu cầu đến thiết kế, thể hiện rõ cách các chức năng cốt lõi vận hành và cách dữ liệu, giao diện, cũng như triển khai được tổ chức. Các sơ đồ Use Case và CSDL giúp làm rõ luồng nghiệp vụ, còn phần UI/UX và DevOps cho thấy hướng hiện thực hóa cụ thể. Đây là nền tảng để bước sang Chương 4, nơi các thành phần sẽ được triển khai và đánh giá trên môi trường thực tế.

CHƯƠNG 6. KẾT QUẢ THỰC NGHIỆM

6.1 Thiết kế kiến trúc

6.1.1 Lựa chọn kiến trúc phần mềm

Mục này có độ dài từ một đến ba trang. Sinh viên cần lựa chọn kiến trúc phần mềm cho ứng dụng của mình như: kiến trúc ba lớp MVC, MVP, SOA, Microservice, v.v. rồi giải thích sơ bộ về kiến trúc đó (không giải thích chi tiết/dài dòng). Sử dụng kiến trúc phần mềm đã chọn ở trên, sinh viên mô tả kiến trúc cụ thể cho ứng dụng của mình. Gợi ý: sinh viên áp dụng lý thuyết chung vào hệ thống/sản phẩm của mình như thế nào, có thay đổi, bổ sung hoặc cải tiến gì không. Ví dụ, thành phần M trong kiến trúc lý thuyết MVC sẽ là những thành phần cụ thể nào (ví dụ: là interface I + class C1 + class C2, v.v.) trong kiến trúc phần mềm của sinh viên.

6.1.2 Thiết kế tổng quan

Sinh viên vẽ biểu đồ gói UML (UML package diagram), nêu rõ sự phụ thuộc giữa các gói (package). SV cần vẽ các gói sao cho chúng được phân theo các tầng rõ ràng, không được sắp đặt package lộn xộn trong hình vẽ. Sinh viên chú ý các quy tắc thiết kế (Các gói không phụ thuộc lẫn nhau, gói tầng dưới không phụ thuộc gói tầng trên, không phụ thuộc bỏ qua tầng, v.v.) và cần giải thích sơ lược về mục đích/nhiệm vụ của từng package. SV tham khảo ví dụ minh họa trong Hình ??



Hình 6.1: Ví dụ biểu đồ phụ thuộc gói

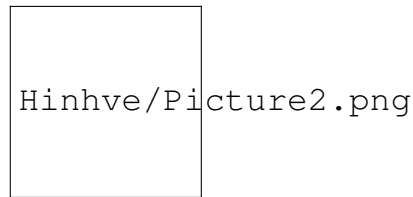
6.1.3 Thiết kế chi tiết gói

Sinh viên thiết kế và lần lượt vẽ biểu đồ thiết kế cho từng package, hoặc một nhóm các package liên quan để giải quyết một vấn đề gì đó. Khi vẽ thiết kế gói, sinh viên chỉ cần đưa tên lớp, không cần chỉ ra các thành viên phương thức và thuộc tính. SV tham khảo ví dụ minh họa trong Hình ??.

Sinh viên cần vẽ rõ ràng quan hệ giữa các lớp trong biểu đồ. Các quan hệ bao gồm: phụ thuộc (dependency), kết hợp (association), kết tập (aggregation), hợp thành (composition), kế thừa (inheritance), và thực thi (implementation). Các quan hệ này đều đã được minh họa trong ??.

Sau khi vẽ hình minh họa, sinh viên cần giải thích ngắn gọn về thiết kế của

mình.



Hình 6.2: Ví dụ thiết kế gói

6.2 Thiết kế chi tiết

6.2.1 Thiết kế giao diện

Phần này có độ dài từ hai đến ba trang. Sinh viên đặc tả thông tin về màn hình mà ứng dụng của mình hướng tới, bao gồm độ phân giải màn hình, kích thước màn hình, số lượng màu sắc hỗ trợ, v.v. Tiếp đến, sinh viên đưa ra các thống nhất/chuẩn hóa của mình khi thiết kế giao diện như thiết kế nút, điều khiển, vị trí hiển thị thông điệp phản hồi, phối màu, v.v. Sau cùng sinh viên đưa ra một số hình ảnh minh họa thiết kế giao diện cho các chức năng quan trọng nhất. Lưu ý, sinh viên không nhầm lẫn giao diện thiết kế với giao diện của sản phẩm sau cùng.

6.2.2 Thiết kế lớp

Phần này có độ dài từ ba đến bốn trang. Sinh viên trình bày thiết kế chi tiết các thuộc tính và phương thức cho một số lớp chủ đạo/quan trọng nhất của ứng dụng (từ 2-4 lớp). Thiết kế chi tiết cho các lớp khác, nếu muốn trình bày, sinh viên đưa vào phần phụ lục.

Để minh họa thiết kế lớp, sinh viên thiết kế luồng truyền thông điệp giữa các đối tượng tham gia cho 2 đến 3 use case quan trọng nào đó bằng biểu đồ trình tự (hoặc biểu đồ giao tiếp).

6.2.3 Thiết kế cơ sở dữ liệu

Phần này có độ dài từ hai đến bốn trang. Sinh viên thiết kế, vẽ và giải thích biểu đồ thực thể liên kết (E-R diagram). Từ đó, sinh viên thiết kế cơ sở dữ liệu tùy theo hệ quản trị cơ sở dữ liệu mà mình sử dụng (SQL, NoSQL, Firebase, v.v.)

6.3 Xây dựng ứng dụng

6.3.1 Thư viện và công cụ sử dụng

Sinh viên liệt kê các công cụ, ngôn ngữ lập trình, API, thư viện, IDE, công cụ kiểm thử, v.v. mà mình sử dụng để phát triển ứng dụng. Mỗi công cụ phải được chỉ rõ phiên bản sử dụng. SV nên kẻ bảng mô tả tương tự như Bảng ???. Nếu có nhiều nội dung trình bày, sinh viên cần xoay ngang bảng.

Mục đích	Công cụ	Địa chỉ URL
IDE lập trình	Eclipse Oxygen a64 bit	http://www.eclipse.org/
v.v.	v.v.	v.v.

Bảng 6.1: Danh sách thư viện và công cụ sử dụng

6.3.2 Kết quả đạt được

Sinh viên trước tiên mô tả kết quả đạt được của mình là gì, ví dụ như các sản phẩm được đóng gói là gì, bao gồm những thành phần nào, ý nghĩa, vai trò?

Sinh viên cần thống kê các thông tin về ứng dụng của mình như: số dòng code, số lớp, số gói, dung lượng toàn bộ mã nguồn, dung lượng của từng sản phẩm đóng gói, v.v. Tương tự như phần liệt kê về công cụ sử dụng, sinh viên cũng nên dùng bảng để mô tả phần thông tin thống kê này.

6.3.3 Minh họa các chức năng chính

Sinh viên lựa chọn và đưa ra màn hình cho các chức năng chính, quan trọng, và thú vị nhất. Mỗi giao diện cần phải có lời giải thích ngắn gọn. Khi giải thích, sinh viên có thể kết hợp với các chú thích ở trong hình ảnh giao diện.

6.4 Kiểm thử

Phần này có độ dài từ hai đến ba trang. Sinh viên thiết kế các trường hợp kiểm thử cho hai đến ba chức năng quan trọng nhất. Sinh viên cần chỉ rõ các kỹ thuật kiểm thử đã sử dụng. Chi tiết các trường hợp kiểm thử khác, nếu muốn trình bày, sinh viên đưa vào phần phụ lục. Sinh viên sau cùng tổng kết về số lượng các trường hợp kiểm thử và kết quả kiểm thử. Sinh viên cần phân tích lý do nếu kết quả kiểm thử không đạt.

6.5 Triển khai

Sinh viên trình bày mô hình và/hoặc cách thức triển khai thử nghiệm/thực tế. Ứng dụng của sinh viên được triển khai trên server/thiết bị gì, cấu hình như thế nào. Kết quả triển khai thử nghiệm nếu có (số lượng người dùng, số lượng truy cập, thời gian phản hồi, phản hồi người dùng, khả năng chịu tải, các thống kê, v.v.)

CHƯƠNG 7. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này có độ dài tối thiểu 5 trang, tối đa không giới hạn.¹ Sinh viên cần trình bày tất cả những nội dung đóng góp mà mình thấy tâm đắc nhất trong suốt quá trình làm ĐATN. Đó có thể là một loạt các vấn đề khó khăn mà sinh viên đã từng bước giải quyết được, là giải thuật cho một bài toán cụ thể, là giải pháp tổng quát cho một lớp bài toán, hoặc là mô hình/kiến trúc hữu hiệu nào đó được sinh viên thiết kế.

Chương này **là cơ sở quan trọng** để các thầy cô đánh giá sinh viên. Vì vậy, sinh viên cần phát huy tính sáng tạo, khả năng phân tích, phản biện, lập luận, tổng quát hóa vấn đề và tập trung viết cho thật tốt. Mỗi giải pháp hoặc đóng góp của sinh viên cần được trình bày trong một mục độc lập bao gồm ba mục con: (i) dẫn dắt/giới thiệu về bài toán/vấn đề, (ii) giải pháp, và (iii) kết quả đạt được (nếu có).

Sinh viên lưu ý **không trình bày lặp lại nội dung**. Những nội dung đã trình bày chi tiết trong các chương trước không được trình bày lại trong chương này. Vì vậy, với nội dung hay, mang tính đóng góp/giải pháp, sinh viên chỉ nên tóm lược/mô tả sơ bộ trong các chương trước, đồng thời tạo tham chiếu chéo tới đề mục tương ứng trong Chương 5 này. Chi tiết thông tin về đóng góp/giải pháp được trình bày trong mục đó.

Ví dụ, trong Chương 4, sinh viên có thiết kế được kiến trúc đáng lưu ý gì đó, là sự kết hợp của các kiến trúc MVC, MVP, SOA, v.v. Khi đó, sinh viên sẽ chỉ mô tả ngắn gọn kiến trúc đó ở Chương 4, rồi thêm các câu có dạng: “Chi tiết về kiến trúc này sẽ được trình bày trong phần 5.1”.

¹Trong trường hợp phần này dưới 5 trang thì sinh viên nên gộp vào phần kết luận, không tách ra một chương riêng rẽ nữa.

CHƯƠNG 8. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

8.1 Kết luận

Sinh viên so sánh kết quả nghiên cứu hoặc sản phẩm của mình với các nghiên cứu hoặc sản phẩm tương tự.

Sinh viên phân tích trong suốt quá trình thực hiện ĐATN, mình đã làm được gì, chưa làm được gì, các đóng góp nổi bật là gì, và tổng hợp những bài học kinh nghiệm rút ra nếu có.

8.2 Hướng phát triển

Trong phần này, sinh viên trình bày định hướng công việc trong tương lai để hoàn thiện sản phẩm hoặc nghiên cứu của mình.

Trước tiên, sinh viên trình bày các công việc cần thiết để hoàn thiện các chức năng/nhiệm vụ đã làm. Sau đó sinh viên phân tích các hướng đi mới cho phép cải thiện và nâng cấp các chức năng/nhiệm vụ đã làm.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Lưu ý: Sinh viên không được đưa bài giảng/slide, các trang Wikipedia, hoặc các trang web thông thường làm tài liệu tham khảo.

Một trang web được phép dùng làm tài liệu tham khảo **chỉ khi** nó là công bố chính thống của cá nhân hoặc tổ chức nào đó. Ví dụ, trang web đặc tả ngôn ngữ XML của tổ chức W3C <https://www.w3.org/TR/2008/REC-xml-20081126/> là TLTK hợp lệ.

Có năm loại tài liệu tham khảo mà sinh viên phải tuân thủ đúng quy định về cách thức liệt kê thông tin như sau. Lưu ý: các phần văn bản trong cặp dấu < > dưới đây chỉ là hướng dẫn khai báo cho từng loại tài liệu tham khảo; sinh viên cần xóa các phần văn bản này trong ĐATN của mình.

<**Bài báo đăng trên tạp chí khoa học:** Tên tác giả, tên bài báo, tên tạp chí, volume, từ trang đến trang (nếu có), nhà xuất bản, năm xuất bản >

hovy1993automated E. H. Hovy, "Automated discourse generation using discourse structure relations," *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993

<**Sách:** Tên tác giả, tên sách, volume (nếu có), lần tái bản (nếu có), nhà xuất bản, năm xuất bản>

peterston2007computer L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.

NguyenThucHai N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.

<**Tập san Báo cáo Hội nghị Khoa học:** Tên tác giả, tên báo cáo, tên hội nghị, ngày (nếu có), địa điểm hội nghị, năm xuất bản>

poesio2001discourse M. Poesio and B. Di Eugenio, "Discourse structure and anaphoric accessibility," in *ESSLLI workshop on information structure, discourse structure and discourse semantics*, Copenhagen, Denmark, 2001, pp. 129–143.

<**Đồ án tốt nghiệp, Luận văn Thạc sĩ, Tiến sĩ:** Tên tác giả, tên đồ án/luận văn, loại đồ án/luận văn, tên trường, địa điểm, năm xuất bản>

knott1996data A. Knott, "A data-driven methodology for motivating a set of coherence relations," Ph.D. dissertation, The University of Edinburgh, UK, 1996.

<**Tài liệu tham khảo từ Internet:** Tên tác giả (nếu có), tựa đề, cơ quan (nếu

có), địa chỉ trang web, thời gian lần cuối truy cập trang web>

BernersTim T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. [Online]. Available: `ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z` (visited on 09/30/2010).

LectureA Princeton University, *Wordnet*. [Online]. Available: `http://www.cogsci.princeton.edu/~wn/index.shtml` (visited on 09/30/2010).

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP

Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đồ án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template ĐATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là “Text Only” để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ ĐATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong ĐATN nhất thiết phải được SV giải thích và tham chiếu tới ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày ĐATN theo kiểu viết ý hoặc gạch đầu dòng. ĐATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/choi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết ĐATN một cách cẩu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi

trong từng trang báo cáo ĐATN.

A.1 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

Đối với kỹ sư chính quy: Từ K61 trở về trước: Ngành Kỹ thuật phần mềm Từ K62 trở về sau: Ngành Khoa học máy tính Đối với cử nhân: Ngành Công nghệ thông tin Đối với chương trình EliteTech: Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin Chương trình ICT Global: Ngành Information Technology Chương trình DS&AI: Ngành Khoa học dữ liệu

A.2 Đánh dấu (bullet) và đánh số (numering)

Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau

```
\begin{itemize}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{itemize}
```

- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau

```
\begin{enumerate}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{enumerate}
```

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại <https://www.overleaf.com/learn/latex/Lists>

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Bảng A.1: Table to test captions and labels.



Hình A.1: Internet vạn vật

A.3 Cách thêm bảng

Bảng ?? là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong Latex khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: <https://www.tablesgenerator.com/>). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong Latex tại link <https://www.overleaf.com/learn/latex/Tables>.

A.4 Chèn hình ảnh

Hình ?? là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong Latex tại

https://www.overleaf.com/learn/latex/Inserting_Images.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận.

A.5 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Ví dụ của việc trích dẫn như sau **scott2013sdn**. Cụ thể, sinh viên sử dụng lệnh `\cite{}` như sau **ashton2009internet**. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.6 Cách viết phương trình và công thức toán học

Các gói `amsmath`, `amssymb`, `amsfonts` hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file `main.tex`. Một ví dụ về tạo phương trình (??) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \quad (\text{A.1})$$

Phương trình ?? là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.7 Qui cách đóng quyển

Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình ?? Phần gáy ĐATN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐATN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:



Hình A.2: Qui cách đóng quyển đồ án



Hình A.3: Qui cách đóng quyển đồ án

B. ĐẶC TẢ USE CASE

Nếu trong nội dung chính không đủ không gian cho các use case khác (ngoài các use case nghiệp vụ chính) thì đặc tả thêm cho các use case đó ở đây.

B.1 Đặc tả use case “Thông kê tình hình mượn sách”

...

B.2 Đặc tả use case “Đăng ký làm thẻ mượn”

...